GitHub Copilot CLI

Kompletny przewodnik po opcjach i programowaniu agentycznym w terminalu

Materiał wykładowy · stan na 6 października 2026 · wersja stabilna 1.0.92

Tomasz Kopacz



Plan wykładu

1. Wprowadzenie: czym jest Copilot CLI, pętla agentyczna
2. Instalacja, aktualizacje i logowanie
3. Podkomendy wiersza poleceń (copilot …)
4. Interfejs interaktywny i skróty klawiszowe
5. Tryby pracy agenta: standard, plan, autopilot, fleet
6. Narzędzia, uprawnienia i bezpieczeństwo
7. Modele, rozumowanie i koszty
8. Kontekst, instrukcje i pamięć
9. Sesje, równoległość i współpraca
10. Serwery MCP
11. Skills, agenci, hooki, wtyczki, LSP
12. Konfiguracja i ustawienia
13. Tryb programistyczny, CI/CD i serwer ACP
14. Wzorce pracy agentycznej, ściągawki i ćwiczenia

Jak czytać ten materiał

Źródła: GitHub Docs (Copilot CLI reference i strony how-to), changelog repozytorium github/copilot-cli, GitHub Changelog — stan na 6.10.2026, wersja 1.0.92	
`copilot --opcja`	opcja wiersza poleceń przy uruchamianiu
`/polecenie`	polecenie slash wpisywane w sesji interaktywnej
`Ctrl+X → v`	sekwencja klawiszy (najpierw Ctrl+X, potem klawisz)
[EXP]	funkcja eksperymentalna: wymaga --experimental lub /experimental on, może się zmienić (poza zakresem RTM)
[ORG]	dostępność zależy od planu Copilot lub polityk organizacji/przedsiębiorstwa
Zawsze aktualna lista: copilot help, copilot help <temat>, /help	

Część 1

01

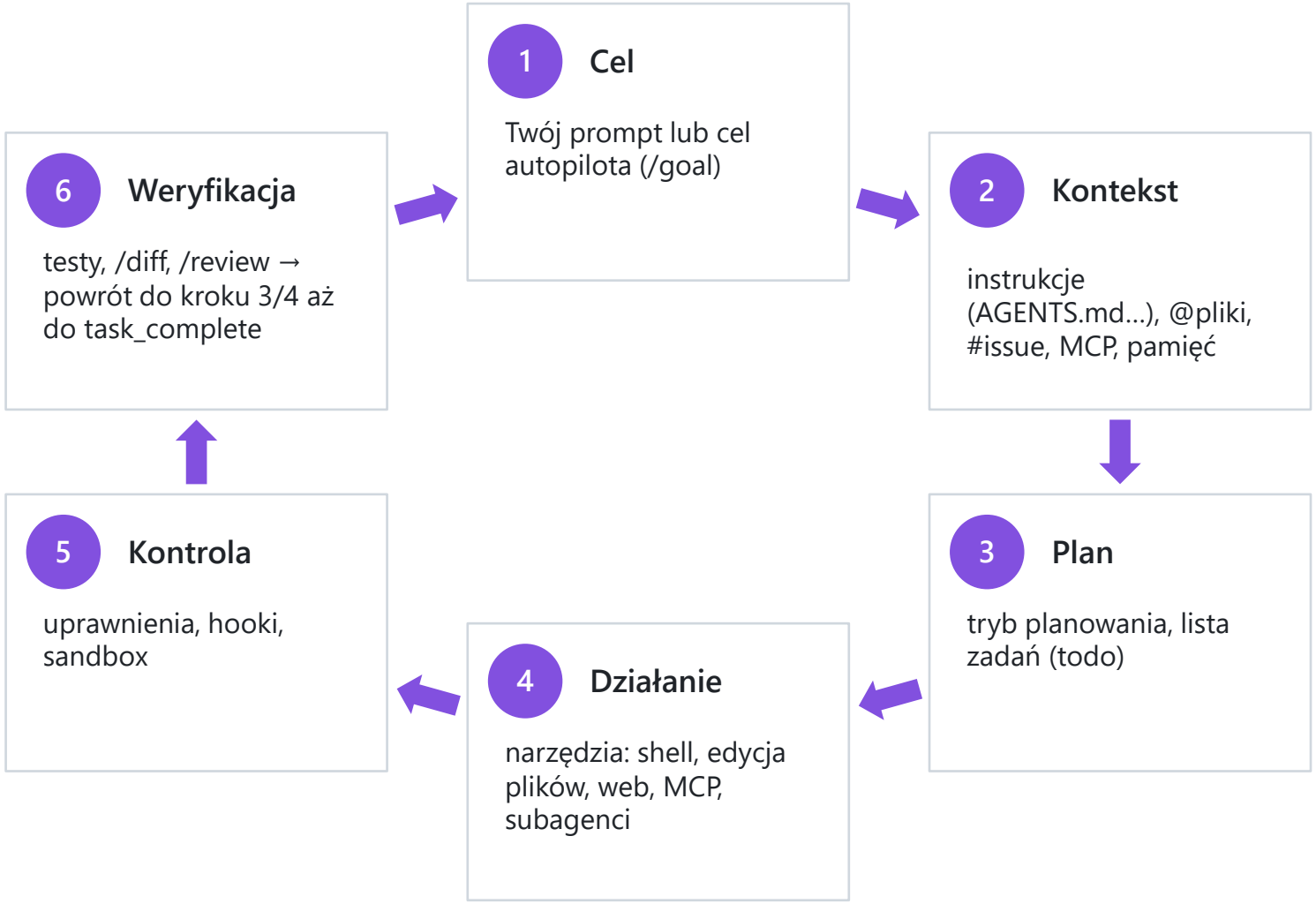
Wprowadzenie

Czym jest Copilot CLI i jak działa pętla agentyczna

Czym jest GitHub Copilot CLI

Agent programistyczny działający natywnie w terminalu — rozmawiasz w języku naturalnym, a agent działa na Twoim kodzie
Ten sam mechanizm agentowy („agentic harness”), który napędza Copilot cloud agent na GitHub
Planuje złożone zadania, edytuje pliki, uruchamia polecenia i testy, iteruje aż do ukończenia
Zna kontekst GitHub: issues, pull requesty, gisty; wbudowany serwer GitHub MCP; delegowanie pracy do chmury
Ty wybierasz poziom autonomii: od zatwierdzania każdej akcji po pełny autopilot
Ogólnie dostępny (GA) od 25 lutego 2026; dostępny we wszystkich planach Copilot — w organizacjach po włączeniu polityki Copilot CLI **[ORG]**

Pętla agentyczna w Copilot CLI

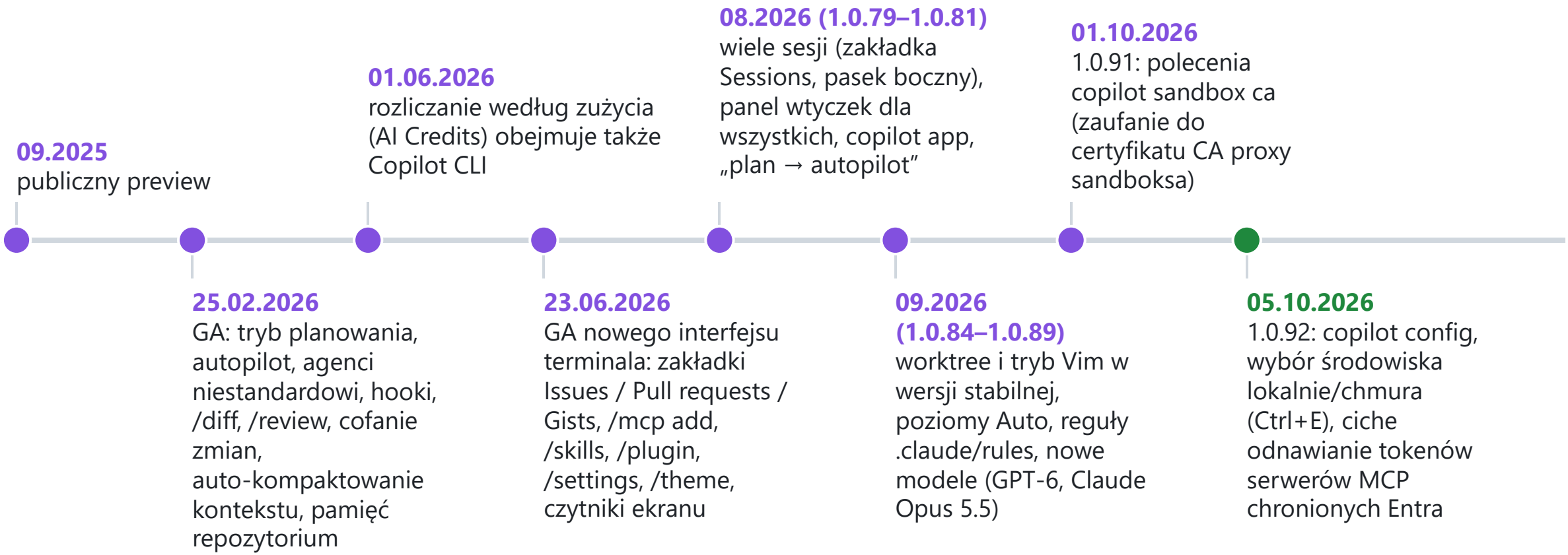


Człowiek w pętli	
Standard	zatwierdzasz użycie narzędzi
Plan	zatwierdzasz plan przed kodowaniem
Autopilot	ustawiasz cel, budżet i granice uprawnień

Mapa możliwości — gdzie szukać opcji

Obszar	Najważniejsze elementy
Tryby pracy	`Shift+Tab`, `/plan`, `/autopilot (/goal)`, `/fleet`, `/delegate`
Narzędzia i uprawnienia	`--allow-tool`, `--deny-tool`, `/permissions`, `--available-tools`, `/sandbox` **[EXP]**
Modele i koszty	`/model`, `--model`, `--effort`, `Auto`, `/usage`, `/limits`, `BYOK`
Kontekst i pamięć	`@plik`, `AGENTS.md`, `/init`, `/context`, `/compact`, `/chronicle`
Sesje	`--resume`, `--continue`, `/fork`, `--worktree`, `/share`, `/remote`
Rozszerzenia	MCP, skills, agenci (.agent.md), hooki, wtyczki, LSP
Automatyzacja	`-p`, `-s`, `--output-format json`, `copilot workflow run`, `--acp`
Integracja z GitHub	#123, zakładki Issues/Pull requests, /pr, /review, /delegate

Kamienie milowe Copilot CLI



Część 2

02

Instalacja, aktualizacje i logowanie

Wymagania, plany i polityki

Obszar	Szczegóły
Plany	Wszystkie plany Copilot: Free, Student, Pro, Pro+, Max, Business, Enterprise
Organizacje [ORG]	Polityka „Copilot CLI" musi być włączona; enterprise: AI controls → Copilot → Features & clients → Clients → Copilot CLI („Enabled everywhere" / „Disabled everywhere" / „Let organizations decide")
`/delegate`	Wymaga polityk Copilot CLI i Copilot cloud agent
Inne polityki	Włączone modele, agenci firmowi, wykluczenia treści, polityki serwerów MCP (rejestr + allowlista), dziennik audytu
System	Linux, macOS, Windows (x64 i arm64; Linux także musl)
Windows	PowerShell 6 lub nowszy (CLI preferuje pwsh 7+)
Instalacja przez npm	Node.js 22 lub nowszy

Instalacja Copilot CLI

Metoda	Wersja stabilna	Kanał prerelease
npm (macOS, Linux, Windows; Node.js 22+)	`npm install -g @github/copilot`	`npm install -g @github/copilot@prerelease`
Homebrew (macOS, Linux)	`brew install --cask copilot-cli`	`brew install --cask copilot-cli@prerelease`
WinGet (Windows)	`winget install GitHub.Copilot`	`winget install GitHub.Copilot.Prerelease`
Skrypt (macOS, Linux)	`curl -fsSL https://gh.io/copilot-install | bash`	zmienna VERSION
Pliki z wydań GitHub	.msi, .zip, .tar.gz z github.com/github/copilot-cli/releases	tagi v1.0.92-N

```bash
# skrypt: konkretna wersja i własny katalog (instalacja do $PREFIX/bin)
curl -fsSL https://gh.io/copilot-install | VERSION="v1.0.92" PREFIX="$HOME/custom" bash
wget -qO- https://gh.io/copilot-install | bash           # alternatywa dla curl
npm_config_ignore_scripts=false npm install -g @github/copilot   # gdy ~/.npmrc ma ignore-scripts=true
```

Aktualizacje i wersje

Polecenie / opcja	Działanie
`copilot update` · `/update` · `/upgrade`	Pobiera i instaluje najnowszą wersję
`copilot version` · `/version`	Pokazuje wersję i sprawdza dostępność aktualizacji
`/changelog [summarize] [WERSJA | last N | since WERSJA]`	Lista zmian (alias /release-notes); summarize = streszczenie AI
`/downgrade WERSJA`	Pobiera wskazaną wersję i restartuje CLI (konta zespołowe)
`--no-auto-update`	Wyłącza automatyczne pobieranie aktualizacji dla tego uruchomienia
`COPILOT_AUTO_UPDATE=false`	Wyłącza auto-aktualizacje CLI i wtyczek first-party (np. w CI)
`/restart`	Restartuje CLI, przywracając wszystkie aktywne sesje procesu

Logowanie: copilot login

Opcja	Działanie
`copilot login`	Uwierzytelnienie OAuth; lokalnie domyślnie przepływ przeglądarkowy
`--web-flow`	Wymusza przepływ przez przeglądarkę (domyślny na lokalnym terminalu)
`--device-code`	Wymusza OAuth device code (domyślny przez SSH, w Codespaces, dev container, CI)
`--with-token`	Czyta token ze standardowego wejścia zamiast OAuth (bez echa w terminalu)
`--host HOST`	Inny host GitHub, np. GitHub Enterprise Cloud z rezydencją danych: https://example.ghe.com
`/login · /logout`	Logowanie / wylogowanie w trakcie sesji
`/user [show | list | switch]`	Pokazuje, listuje i przełącza konta GitHub

Tokeny i środowiska bez przeglądarki

Kolejność odczytu tokenu ze zmiennych: COPILOT_GITHUB_TOKEN → GH_TOKEN → GITHUB_TOKEN	
Obsługiwane: fine-grained PAT (v2) z uprawnieniem „Copilot Requests” (token użytkownika — bez wybierania organizacji jako właściciela), tokeny OAuth aplikacji Copilot CLI oraz GitHub CLI (gh)	
NIE są obsługiwane klasyczne PAT (ghp_…)	
`GH_HOST`	host GitHub dla CLI (np. GHE.com); COPILOT_GH_HOST — nadpisuje host tylko dla Copilot CLI
`storeTokenPlaintext`	zgoda na zapis tokenu tekstem jawnym, gdy brak systemowego magazynu poświadczeń

```bash
# CI / automatyzacja: fine-grained PAT z uprawnieniem "Copilot Requests"
export COPILOT_GITHUB_TOKEN=github_pat_...
copilot -p "Podsumuj zmiany w tym repozytorium" -s

# token z pliku, bez przepływu OAuth
copilot login --with-token < mytoken.txt
```

Pierwsze uruchomienie i pomoc

Uruchom copilot w katalogu projektu → CLI zapyta, czy ufasz temu folderowi	
Brak instrukcji repozytorium? CLI podpowie /init (wygenerowanie .github/copilot-instructions.md)	
`--banner / --no-banner`	pokaż / ukryj animowany baner startowy
`copilot help [TEMAT]`	`tematy: billing, config, commands, environment, logging, monitoring, permissions, providers, sandbox`
`copilot completion bash|zsh|fish`	skrypt uzupełniania podkomend, opcji i wartości

```bash
source <(copilot completion bash)                          # bash, bieżąca sesja
copilot completion bash | sudo tee /etc/bash_completion.d/copilot  # bash, na stałe
copilot completion zsh  > "${fpath[1]}/_copilot"            # zsh
copilot completion fish > ~/.config/fish/completions/copilot.fish  # fish
```

Część 3

03

Podkomendy wiersza poleceń

Wszystko, co można zrobić bez otwierania sesji

Podkomendy copilot (1/2)

Podkomenda	Do czego służy
`copilot`	Uruchamia interaktywny interfejs (TUI)
`copilot app`	Otwiera aplikację GitHub Copilot w bieżącym katalogu (nowa sesja przez deep link)
`copilot completion SHELL`	Skrypt uzupełniania dla bash, zsh, fish
`copilot help [TEMAT]`	Pomoc ogólna lub tematyczna
`copilot init`	Generuje instrukcje Copilot dla repozytorium (.github/copilot-instructions.md)
`copilot login [OPCJA]`	Uwierzytelnienie (OAuth, device code, token)
`copilot mcp …`	Zarządzanie serwerami MCP bez sesji (list, get, add, enable, disable, remove)
`copilot plugin …`	Wtyczki i marketplace'y (alias historyczny: copilot plugins)

Podkomendy copilot (2/2)

Podkomenda	Do czego służy
`copilot instruction list [--json]`	Lista wykrytych źródeł instrukcji dla bieżącego katalogu
`copilot lsp list [--json]`	Lista skonfigurowanych serwerów językowych (tylko podgląd)
`copilot skill …`	Skills: list, add, remove, enable, disable
`copilot sandbox ca status|create|trust|rotate|remove`	Certyfikat CA proxy sandboksa (np. provisioning bez TTY)
`copilot config …`	Listowanie, odczyt, ustawianie i usuwanie ustawień (nowość w 1.0.92)
`copilot workflow run NAZWA`	Uruchamia dynamiczny workflow bez promptu czatu **[EXP]**
`copilot update · copilot version`	Aktualizacja · wersja i sprawdzenie aktualizacji

Inspekcja środowiska bez sesji

--json daje wynik maszynowy — idealny do audytu konfiguracji stanowisk
`copilot instruction list --json → wpisy { id, label, description?, location, type, sourcePath, defaultDisabled, applyTo? }`
`copilot lsp list --json → wpisy { id, fileExtensions?, sourcePlugin? }`
Uwaga: lista instrukcji z wtyczek uwzględnia tylko globalne ustawienia użytkownika — sesja w zaufanym repo może zastosować więcej
--config-dir jest przestarzałe → używaj COPILOT_HOME
W sesji pełny obraz daje /env (instrukcje, MCP, skills, agenci, hooki, wtyczki, LSP, rozszerzenia)

```bash
copilot instruction list --json | jq '.[] | {label, sourcePath}'
copilot mcp list --json
copilot lsp list
```

Część 4

04

Interfejs interaktywny

Prompt, oś czasu, zakładki, sesje i skróty klawiszowe



Anatomia ekranu sesji

Zakładki u góry: bieżąca sesja, Sessions (lista sesji), Gists; w repozytorium GitHub także Issues i Pull requests — Tab / Shift+Tab lub mysz
Na zakładkach Issues/PR: c wstawia odwołanie do promptu, o otwiera element w przeglądarce, / wyszukuje w GitHub
Oś czasu (timeline): odpowiedzi, wywołania narzędzi, wyniki poleceń na żywo
Pole promptu + kolejka oczekujących wiadomości („pending”)
Panel celu autopilota (cel, zużyte kredyty, postęp todo) — Ctrl+X → g
Pasek boczny sesji (←) i stopka/status line (/statusline, alias /footer)

Składnia promptu

Zapis	Znaczenie
`@ŚCIEŻKA`	Dołącza zawartość pliku do kontekstu
`#NUMER`	Dołącza issue lub pull request z GitHub
`!POLECENIE`	Wykonuje polecenie w lokalnym shellu z pominięciem Copilota
`! (sam)`	Tryb shell — wiele poleceń po kolei; wyjście: Esc lub Ctrl+C na pustym prompcie
`$ (sam) + Enter`	Przekazuje terminal prawdziwemu shellowi ($SHELL / %COMSPEC%); powrót: exit
`/POLECENIE`	Polecenie slash
`? (na pustym prompcie)`	Szybka pomoc; drugie ? wstawia znak dosłownie
`Ctrl+X → /`	Uruchom polecenie slash bez kasowania wpisanego promptu

Sterowanie agentem w trakcie pracy

Klawisz	Działanie
`Esc`	Anuluje bieżącą operację
`Esc ×2`	Przerywa trwającą turę; gdy agent główny bezczynny — zatrzymuje agentów w tle; jeśli model jeszcze nie zaczął odpowiadać — oddaje prompt do edycji
`Ctrl+C / ×2`	Anuluje lub czyści wpis / zamyka CLI
`Ctrl+D`	Zamyka CLI
`Ctrl+Q lub Ctrl+Enter`	Kolejkuje wiadomość, gdy agent jest zajęty (wpisy „pending” można anulować)
`↑ (pusty prompt)`	Przywraca ostatni zakolejkowany lub sterujący prompt do edycji
`Ctrl+P`	Historia promptów bez przywoływania kolejki
`Ctrl+X → b`	Przenosi bieżące zadanie lub polecenie shell do tła

Skróty globalne

Skrót	Działanie
`Shift+Tab`	Przełącza tryb: standard → plan → autopilot
`Ctrl+G lub Ctrl+X → e`	Edycja promptu w zewnętrznym edytorze ($EDITOR)
`Ctrl+L`	Czyści ekran
`Ctrl+R`	Wyszukiwanie wstecz w historii poleceń
`Ctrl+V / Alt+V`	Wkleja schowek jako załącznik / wkleja obraz
`Ctrl+Space lub Ctrl+X → v`	Dyktowanie głosowe wł./wył.
`Ctrl+X → o`	Otwiera ostatni link z osi czasu
`Ctrl+X → h / Ctrl+X → x`	Ukrywa pasek boczny / zamyka bieżącą sesję
`Shift+Enter (Alt/Option+Enter)`	Nowa linia w prompcie
`Ctrl+Z`	Zawiesza proces (Unix)

Oś czasu i edycja promptu

Oś czasu (gdy prompt jest pusty)	
`Ctrl+F`	wyszukiwanie na osi czasu (też /search, /find)
`Ctrl+O`	rozwija ostatnie elementy
`Ctrl+E`	rozwija wszystkie elementy
`Ctrl+T`	pokazuje / ukrywa rozumowanie modelu
`Page Up` / `Page Down`	przewijanie stronami

Edycja promptu	
`Ctrl+A` / `Ctrl+E`	początek / koniec linii
`Ctrl+B` / `Ctrl+F`	znak w lewo / w prawo
`Ctrl+W` / `Ctrl+U` / `Ctrl+K`	usuń słowo / do początku / do końca linii
Alt+← / Alt+→ (Option na Mac)	skok o słowo
`Ctrl+Home` / `Ctrl+End`	początek / koniec tekstu
`Tab lub Ctrl+Y`	akceptuje podpowiedź inline
`/vim`	modalna edycja w stylu Vim

Personalizacja interfejsu

Polecenie / opcja	Działanie
`/theme [default|github|dim|high-contrast|colorblind]`	Tryb kolorów
`/vim (ustawienie editorMode)`	Tryb Vim w polu promptu
`/statusline (/footer)`	Wybór elementów paska stanu
`/terminal-setup`	Konfiguracja terminala dla Shift+Enter i Ctrl+Enter
`--mouse[=on|off], --no-mouse`	Obsługa myszy w TUI (off = natywne zaznaczanie tekstu)
`--no-color · --screen-reader`	Bez kolorów · optymalizacje dla czytników ekranu
`--plain-diff (PLAIN_DIFF=true)`	Wyłącza bogate renderowanie diffów
`--stream=on|off`	Strumieniowanie odpowiedzi (domyślnie on)
`/voice [on|off|models|devices]`	Tryb głosowy, modele mowy, wybór mikrofonu
`/keep-alive [on|off|busy|CZAS] (/caffeinate)`	Blokuje usypianie komputera

Pomoc i diagnostyka w sesji

Polecenie	Działanie
`/help · ?`	Pomoc dla poleceń interaktywnych · szybka ściągawka
`/env`	Załadowane środowisko: instrukcje, MCP, skills, agenci, hooki, wtyczki, LSP, rozszerzenia
`/diagnose [PROMPT]`	Analiza logu sesji pod kątem błędów i nieoczekiwanych zachowań
`/usage`	Statystyki sesji: tokeny per model, zużycie AI Credits
`/context`	Wizualizacja zajętości okna kontekstu
`/feedback (/bug)`	Zgłoszenie opinii lub błędu
`/changelog …`	Lista zmian CLI
`/search, /find`	Przeszukiwanie osi czasu rozmowy

Część 5

05

Tryby pracy agenta

Standard, plan, autopilot, fleet i agenci wyspecjalizowani



Trzy tryby pracy (Shift+Tab)

Tryb	Jak działa	Kiedy używać	Uruchomienie
Standard (interactive)	Agent działa krok po kroku i pyta o zgodę na narzędzia	Praca eksploracyjna, ryzykowne zmiany	domyślny; --mode=interactive
Plan	Analiza, pytania doprecyzowujące, plan wdrożenia przed kodem	Większe funkcje, refaktoryzacja	`Shift+Tab, /plan [PROMPT], --plan, --mode=plan`
Autopilot	Agent kontynuuje sam aż do task_complete	Dobrze opisane zadania z jasnym celem	`Shift+Tab, /autopilot, /goal, --autopilot, --mode=autopilot`

Tryb planowania i „plan → autopilot”

`/plan [PROMPT] lub Shift+Tab`	agent analizuje, zadaje pytania doprecyzowujące i tworzy ustrukturyzowany plan przed kodem
Po zatwierdzeniu planu do wyboru m.in.: „Accept plan and build on autopilot”, „Accept plan and build on autopilot + /fleet”, „Exit plan mode and I will prompt myself”	
`/session plan`	podgląd planu bieżącej sesji; Ctrl+E rozwija kartę zatwierdzania planu
--plan = skrót --mode plan (nie łączy się z --autopilot)	
`--plan --mode autopilot`	start w planie, automatyczne przejście w autopilota, gdy plan jest gotowy (bez czekania na akceptację)
`COPILOT_PLAN_THEN_AUTOPILOT=1`	to samo dla środowisk, które ustawiają tylko zmienne (jawne opcje mają pierwszeństwo)

```bash
copilot --plan                      # start w trybie planowania
copilot --plan --mode autopilot     # plan, potem autopilot bez czekania na akceptację
copilot -i "/plan Dodaj cache Redis do warstwy API"
```

Autopilot: /autopilot i /goal

`/autopilot [CEL] = /goal [CEL]`	włącza autopilota; bez celu agent przyjmuje za cel ostatni prompt (widoczny w panelu statusu)
Przy wejściu w autopilota CLI pyta o uprawnienia: „Enable all permissions (recommended)" / „Continue with limited permissions" (żądania wymagające zgody są wtedy automatycznie odrzucane) / „Cancel (Esc)"	
`/goal CEL --max-ai-credits N`	budżet AI Credits na cel; po jego wyczerpaniu autopilot pauzuje i pokazuje panel; /goal --max-ai-credits N wznawia z nowym pełnym budżetem
`/goal on / /goal off`	przełącza autopilota bez ustawiania celu; panel celu (cel, kredyty, postęp todo) — Ctrl+X → g
`--autopilot`	praca aż do task_complete; --max-autopilot-continues=N — limit automatycznych kontynuacji (ustawiaj jawnie)
stayInAutopilot (domyślnie true) — czy po zakończeniu zadania zostać w trybie autopilota	
Autopilot zatrzymuje się, gdy: zadanie jest skończone, pojawi się blokujący problem, naciśniesz Ctrl+C lub osiągnięty zostanie limit	

```text
/goal Zrefaktoryzuj moduł auth na JWT; testy muszą przechodzić --max-ai-credits 50
```

```bash
copilot --autopilot --max-autopilot-continues 10 -p "Napraw wszystkie błędy lintera" --allow-tool='write, shell(npm run lint:*)'
```

Subagenci i /fleet — praca równoległa

`/fleet [PROMPT]`	agent główny dzieli zadanie na podzadania i jako orkiestrator uruchamia subagentów równolegle (tam, gdzie pozwalają zależności)
Każdy subagent ma własne okno kontekstu; domyślnie używa taniego modelu — w prompcie można wskazać model albo agenta (@NAZWA-AGENTA)	
`--fleet`	z wiersza poleceń; wymaga -p, -i lub promptu z potoku; nieobsługiwane w trybie serwera ACP
Subagenci mogą tworzyć własnych subagentów (drzewo zadań w /tasks)	
Limity: COPILOT_SUBAGENT_MAX_CONCURRENT (domyślnie 32, zakres 1–256), COPILOT_SUBAGENT_MAX_DEPTH (domyślnie 4, zakres 1–128); ustawienia subagents.maxConcurrency / subagents.maxDepth (dla rozliczeń usage-based)	
`/subagents (/agents)`	modele subagentów; modelPolicy: required (bez nadpisywania) lub preferred

```bash
copilot --fleet -p "Zrefaktoryzuj pakiet utils i zaktualizuj wszystkie miejsca wywołań" --allow-tool write
echo "Zaktualizuj zależności projektu i uruchom testy" | copilot --fleet
copilot --fleet -i "Dodaj testy jednostkowe do każdego modułu w katalogu src"
```

/tasks — podgląd subagentów i poleceń w tle

Klawisz	Działanie
`↑ / K, ↓ / J`	Wybór zadania
`Enter`	Szczegóły zadania lub „teleport” do widoku sesji wybranego subagenta
`a`	Wszystkie poziomy zagnieżdżenia / tylko bieżący poziom
`f`	Pokaż także zakończonych subagentów i polecenia shell
`X`	Zabija wybrane aktywne zadanie
`R`	Usuwa wybrane zakończone zadanie
`B`	Przenosi zadanie synchroniczne do tła
`Esc`	Zamyka okno (lub wraca do listy)

Wbudowani agenci wyspecjalizowani — polecenia

Polecenie	Co robi	Typowe użycie
`/review [PROMPT]`	Agent code review analizuje zmiany; można zawęzić ścieżką lub wzorcem plików	/review src/auth/** przed commitem
`/security-review [PROMPT]`	Przegląd bezpieczeństwa bieżących lokalnych zmian: priorytetyzowane podatności + rekomendacje	Przed PR; to NIE jest pełny audyt repozytorium
`/rubber-duck [PROMPT]`	Krytyk na modelu innej rodziny niż sesja; zwraca: problemy blokujące, nieblokujące, sugestie	`/rubber-duck Jakich przypadków brzegowych brakuje?`
`/research TEMAT`	Badanie kodu, repozytoriów GitHub i web → raport Markdown z cytatami	`/research Jak React realizuje concurrent rendering?`
`/ask PYTANIE (/btw)`	Pytanie poboczne bez narzędzi; nie trafia do historii; działa także w trakcie pracy agenta	`/ask Co oznacza „idempotentny” w poprzedniej odpowiedzi?`

Wbudowani subagenci i ich ustawienia

Subagent	Rola
`explore`	Szybka, lekka eksploracja kodu (grep, glob, view, shell); niczego nie zmienia, więc może działać równolegle
`task`	Polecenia deweloperskie (testy, build, lintery, instalacje): krótkie podsumowanie przy sukcesie, pełny wynik przy błędzie
`general-purpose`	Prawie wszystkie możliwości agenta głównego w osobnym oknie kontekstu
`code-review`	Przegląd zmian i diffów gałęzi z wysokim stosunkiem sygnału do szumu; bez uwag o stylu (/review)
`security-review`	Tylko do odczytu; zgłasza wyłącznie pewne, wykorzystywalne podatności (/security-review)
`research`	Badania z wyszukiwaniem GitHub i web — uruchamiany tylko przez /research
`rubber-duck`	Krytyk planów, kodu i testów na modelu innej rodziny (/rubber-duck)

Ustawienia
`subagents.disabledSubagents`
wyłączenie wybranych subagentów (wszystkich poza rubber-duck)
`builtInAgents.rubberDuck (domyślnie true), builtInAgents.rubberDuckAutoInvoke (domyślnie false)`
`subagents.agents`
model, modelPolicy, effortLevel, contextTier per agent (edytor: /subagents)

/delegate — praca w chmurze GitHub

/delegate [PROMPT] lub prefiks & na początku promptu — zadanie trafia do Copilot cloud agent na GitHub
Przebieg: commit niezatwierdzonych zmian jako punkt kontrolny w nowej gałęzi → szkic pull requesta → praca w tle → prośba o Twój review
Praca trwa nawet po wyłączeniu komputera; CLI wypisuje linki do PR i sesji agenta
Sesje w chmurze widać w zakładce Sessions — można je wznowić i sterować lokalnie
Ctrl+E przed rozpoczęciem rozmowy — wybór środowiska: lokalnie lub w chmurze (1.0.92)
Z BYOK: /delegate wymaga zalogowania do GitHub (zadanie wykonuje Copilot GitHub, nie Twój dostawca)

```text
/delegate Uzupełnij testy integracyjne API i napraw failujące przypadki brzegowe
& dodaj walidację danych wejściowych do endpointu rejestracji
```

/pr — pull request bieżącej gałęzi

Podkomenda	Działanie	Commit i push
`/pr lub /pr view · /pr view web`	Status PR · otwarcie PR w przeglądarce	nie
`/pr create`	Tworzy lub aktualizuje PR; wypełnia szablon PR repozytorium	tak
`/pr fix feedback`	Odpowiada na komentarze review i je rozwiązuje	tak
`/pr fix conflicts`	Synchronizuje z gałęzią bazową, rozwiązuje konflikty (mergeStrategy: rebase lub merge)	tak
`/pr fix ci`	Diagnozuje i naprawia failujące checki CI (w pętli)	tak
`/pr fix (/pr fix all)`	feedback → konflikty → CI	tak
`/pr auto`	Tworzy PR w razie potrzeby i poprawia w pętli aż do zielonego; nie merguje	tak
`/pr automerge (/pr agentmerge)`	Jak auto, a po zielonym włącza auto-merge w GitHub	tak

```text
/pr create dodaj do tytułu prefiks "Projekt X: "
/pr fix ci skup się na testach
```

/diff — przegląd zmian i komentarze dla agenta

Klawisz	Działanie
`↑/k, ↓/j`	Poprzednia / następna linia
`←/h, →/l`	Poprzedni / następny plik
`Home/g, End/G`	Pierwsza / ostatnia linia
`PgUp/PgDn, Ctrl+U/Ctrl+D`	Strona / pół strony
`c`	Dodaj lub edytuj komentarz do wybranej linii
`s`	Podsumowanie komentarzy
`Enter`	Wyślij wszystkie komentarze do agenta
`b`	Zmiany niezatwierdzone ↔ diff gałęzi
`w`	Ukryj zmiany samych białych znaków
`r / Esc`	Odśwież (tylko sesje zdalne) / wyjście

Cofanie zmian: /undo i /rewind

/undo lub /rewind — lista punktów cofnięcia (najnowsze pierwsze): początek promptu, wiek, dodane/usunięte linie
„Conversation only” — cofa tylko rozmowę, pliki zostają bez zmian
„Conversation + files” — przywraca też pliki zmienione w tej i późniejszych turach (z podglądem per plik); pomija pliki edytowane przez Ciebie i zbyt duże do kopii
Cofnięty prompt wraca do pola edycji — można go poprawić i wysłać ponownie
Działa bez Git: zmiany śledzone per tura dla edycji, poleceń shell i subagentów
Ograniczenia: nie przed pierwszym promptem, nie w trakcie pracy agenta, nie dla sesji zdalnych; cofnięcia nie da się cofnąć
/session checkpoints [n] — punkty kontrolne tworzone m.in. przy kompaktowaniu kontekstu

Część 6

06

Narzędzia, uprawnienia i bezpieczeństwo

Co agent może zrobić — i kto o tym decyduje



Model zaufania Copilot CLI

Zaufanie do folderu przy starcie: „1. Yes” (tylko ta sesja), „2. Yes, and remember this folder for future sessions”, „3. No (Esc)” (kończy sesję)
Zaufane foldery: ~/.copilot/config.json (tablica trustedFolders); dopiero zaufany katalog ładuje swoje skills, wtyczki, MCP, hooki i LSP
Ścieżki: domyślnie katalog roboczy z podkatalogami oraz katalog tymczasowy; URL-e: każdy wymaga zgody (HTTP i HTTPS osobno)
Zgoda na narzędzie: jednorazowo, do końca sesji albo na stałe dla repozytorium/katalogu (zapis w ~/.copilot/permissions-config.json)
URL zatwierdzony na stałe trafia do allowedUrls w ~/.copilot/settings.json
--allow-tool / --deny-tool działają tylko w bieżącej sesji; reguły odmowy wygrywają z zapamiętanymi zgodami
W trybie -p nie ma pytań — wszystko, co nie jest dozwolone z góry, zostaje odrzucone

Tryby uprawnień

Polecenie / opcja	Działanie
`/permissions default`	Domyślnie: pytanie o zgodę przy narzędziach wymagających akceptacji
`/permissions assisted (/allow-all auto)`	Zgody wspomagane przez LLM: model ocenia bezpieczeństwo każdego żądania; akceptowalne mogą być zatwierdzane automatycznie
`/permissions allow-all (/allow-all, /yolo)`	Wszystkie uprawnienia: narzędzia, ścieżki, URL-e (/allow-all off wyłącza)
`/permissions show · /permissions reset`	Bieżący tryb · reset zgód udzielonych w tej sesji
`/reset-allowed-tools`	Cofa zgody sesji i usuwa zapamiętane zgody dla bieżącej lokalizacji
`--allow-all = --yolo`	`= --allow-all-tools + --allow-all-paths + --allow-all-urls`
`--allow-all-tools`	Wszystkie narzędzia bez pytania (zmienna: COPILOT_ALLOW_ALL)
`defaultPermissionMode (ustawienie)`	Tryb startowy: manual (domyślnie), assisted, allow-all
`COPILOT_ALLOW_ALL=true`	Jak --allow-all; dokładnie true dodatkowo ufa katalogowi i ładuje jego skills, wtyczki, MCP i hooki

Reguły narzędzi: Rodzaj(argument)

Rodzaj	Co kontroluje	Przykłady reguł
`shell`	Wykonywanie poleceń	`shell`, `shell(npm test)`, `shell(git push)`, `shell(git:*)`
`write`	Tworzenie i modyfikacja plików	`write`, `write(README.md)`, `write(.github/copilot-instructions.md)`
`read`	Odczyt plików i katalogów	`read`, `read(.env)`
`url`	Dostęp do URL (web_fetch lub shell)	`url(github.com)`, `url(https://*.github.com)`, `url(http://localhost:3000)`, `url(https://docs.github.com/copilot/*)`
`memory`	Zapis nowych faktów do pamięci agenta	`memory`
`NAZWA-SERWERA`	Narzędzia serwera MCP	`github(create_issue)`, `MyMCP`

Zasady
Brak argumentu = wszystkie narzędzia danego rodzaju
Reguła odmowy zawsze wygrywa z regułą zezwalającą — nawet przy --allow-all
shell(git:*) = „git” + spacja: pasuje do git push, git pull, ale nie do gitea
write(README.md) = dokładna ścieżka lub końcowy segment (*/README.md); dowiązania i ./.. są rozwiązywane, wielkość liter ignorowana na macOS/Windows
Wildcardy tylko: :* w shell, *. na początku hosta i /* na końcu ścieżki w url

Reguły w praktyce

```bash
# wszystkie polecenia git oprócz git push
copilot --allow-tool='shell(git:*)' --deny-tool='shell(git push)'

# jedno narzędzie z serwera MCP / wszystkie narzędzia serwera
copilot --allow-tool='MyMCP(create_issue)'
copilot --allow-tool='MyMCP'

# blokada zapisu konkretnego pliku
copilot --deny-tool='write(secret.txt)'

# automatyzacja: zapis plików + tylko testy, lokalny serwer deweloperski
copilot -p "Napraw failujące testy w module payments" \
  --allow-tool='write, shell(npm test), url(http://localhost:3000)' \
  --deny-tool='shell(rm:*), shell(git push)'
```

Dostępność narzędzi: co model w ogóle widzi

Grupa	Nazwy narzędzi (dla --available-tools / --excluded-tools)
Shell	`bash / powershell (wykonanie), list_bash / list_powershell, read_bash / read_powershell, write_bash / write_powershell, stop_bash / stop_powershell`
Pliki	view, create, edit, apply_patch (część modeli zamiast edit/create)
Agenci i zadania	task (uruchamia subagenta), list_agents, read_agent, write_agent
Inne	`ask_user, glob, grep (lub rg), skill, web_fetch`

Zasady
`--available-tools=…`
biała lista: model widzi tylko wymienione narzędzia
`--excluded-tools=…`
czarna lista: wymienione narzędzia znikają
Różnica: dostępność decyduje, czy model zna narzędzie; uprawnienia (--allow-tool/--deny-tool) — czy może go użyć bez pytania
--no-ask-user = wyłączenie ask_user

Ścieżki, katalogi i adresy URL

Opcja / polecenie	Działanie
`-C KATALOG`	Zmienia katalog roboczy przed wszystkim innym
`--add-dir=ŚCIEŻKA` · `/add-dir ŚCIEŻKA`	Dostęp do dodatkowego katalogu + ładuje jego .github/skills i .github/agents jako zaufane (opcję można powtarzać; katalog musi istnieć)
`/list-dirs`	Lista katalogów z dostępem
`--allow-all-paths`	Wyłącza weryfikację ścieżek — dostęp do dowolnej ścieżki
`--disallow-temp-dir`	Blokuje automatyczny dostęp do katalogu tymczasowego systemu
`--allow-url=URL` · `--deny-url=URL`	Zezwala / zabrania dostępu do adresów lub domen (deny wygrywa)
`--allow-all-urls`	Wszystkie URL-e bez pytania
`/cwd, /cd [ŚCIEŻKA]`	Pokazuje lub zmienia katalog roboczy w sesji

Sekrety i środowisko poleceń

Opcja / zmienna	Działanie
`--secret-env-vars=VAR,…`	Maskuje wartości wskazanych zmiennych w środowisku poleceń shell i serwerów MCP oraz w wynikach
(domyślnie)	Wartości GITHUB_TOKEN i COPILOT_GITHUB_TOKEN są zawsze maskowane
`--bash-env / --no-bash-env`	Włącza / wyłącza obsługę BASH_ENV w powłoce bash
Sandbox (1.0.92)	Powłoki w sandboksie nie dostają GITHUB_TOKEN z otoczenia, chyba że skonfigurowano to jawnie
`COPILOT_CHILD_OOM_SCORE_ADJ`	Linux: priorytet OOM-killera dla procesów uruchamianych przez agenta (domyślnie 300, off = wyłącz)

Sandbox systemowy [EXP]

Element	Działanie
`/sandbox lub /sandbox config`	Okno ustawień: zakładki General, Auth, Filesystem, Network
`/sandbox status · /sandbox policy`	Czy włączony · efektywna polityka: ścieżki wg źródła, dostęp sieciowy, wykryte narzędzia
`/sandbox enable · /sandbox disable`	Włączenie / wyłączenie
`--sandbox / --no-sandbox`	Tylko dla tej sesji, bez zmiany zapisanych ustawień (przydatne z -p)
`/sandbox ca create|trust|rotate|remove`	Certyfikat CA proxy sandboksa
`copilot sandbox ca status|create|trust [CA.PEM]|rotate|remove`	To samo poza sesją (np. provisioning) — uruchamiać jako użytkownik, nie root/SYSTEM

Co obejmuje
polecenia shell, lokalne serwery MCP/LSP, wbudowane narzędzia plikowe i web; zdalne serwery MCP nigdy nie są w sandboksie (status connected (sandboxed) dla lokalnych)
Domyślnie
zapis tylko w katalogu roboczym i tymczasowym; katalog domowy i systemowy tylko do odczytu; reszta zablokowana

Zabezpieczenia firmowe (managed settings)

Ustawienie	Efekt
`permissions.disableBypassPermissionsMode: "disable"`	`Blokuje --allow-all-tools, --allow-all-paths, --allow-all-urls, --allow-all, --yolo i /permissions allow-all`
`permissions.disableBypassPermissionsMode: "allow-auto-only"`	Blokuje allow-all, ale zezwala na /permissions assisted
Nierozpoznana wartość	Fail-closed: działa jak "disable"
`sandbox.enabled + sandbox.allowBypass`	Wymuszenie sandboksa; czy użytkownik może go pominąć w sesji
`sandbox.failIfUnavailable: true`	Gdy sandboksa nie da się zestawić — blokada modelu i narzędzi zamiast pracy bez izolacji
Nieczytelna polityka na urządzeniu	Fail-closed: najostrzejszy poziom sandboksa

Źródła ograniczeń (rosnąca trwałość)	ustawienia użytkownika (~/.copilot/settings.json, maszyna) → ustawienia zarządzane z serwera (konto) → polityka MDM (plist/rejestr/plik, urządzenie)

Część 7

07

Modele, rozumowanie i koszty

Wybór modelu

/model (/models) — wybór modelu, wysiłku rozumowania i okna kontekstu; opcja Auto
Zasięg zmiany: domyślnie / --session (-s) — tylko ta sesja; --repo / --local — ustawienia repozytorium; --global (lub /config model) — domyślny dla przyszłych sesji
W selektorze: Tab — wariant z długim kontekstem; Shift+Tab — grupowanie (polecane / dostawca / kategoria); baner o zasadach retencji danych dostawcy
Zmiana w trakcie tury jest kolejkowana i stosowana po jej zakończeniu (Ctrl+C anuluje)
Pierwszeństwo: model z definicji agenta → --model → COPILOT_MODEL → model w settings.json → domyślny CLI

```bash
copilot --model claude-opus-5.5
COPILOT_MODEL=gpt-5.3-codex copilot -p "Wyjaśnij moduł billing" -s
```

```text
/model --repo claude-sonnet-4.6
/model auto balance
```

Modele w dokumentacji (stan: październik 2026)

Identyfikator	Zastosowanie wg dokumentacji
`claude-sonnet-4.6`	Programowanie ogólnego przeznaczenia (domyślny)
`claude-opus-5.5`	Nowy model o wysokich możliwościach — złożone zadania
`claude-haiku-4.5`	Szybkie, lekkie operacje
`gpt-5.4`	Złożone rozumowanie
`gpt-5.3-codex`	Zadania skoncentrowane na kodzie
`gpt-6-astra`, `gpt-6-sol`, `gpt-6-luna`	Nowe modele — włączane świadomie (nie są domyślne w Auto)
`gemini-3.7-flash`	Szybkie odpowiedzi Google Gemini
`auto`	Copilot sam wybiera najlepszy dostępny model

Auto, wysiłek rozumowania i okno kontekstu

Opcja	Działanie
`--model=auto ("model": "auto")`	Automatyczny wybór modelu — na planach płatnych 10% zniżki na koszt modeli
`/model auto efficiency|balance|intelligence`	Poziom Auto: koszt / równowaga kosztu, jakości i opóźnienia / jakość dla złożonych zadań
`autoTier (ustawienie)`	Domyślny poziom Auto dla nowych rozmów (także jako polityka firmowa)
`--effort=POZIOM / --reasoning-effort=POZIOM`	low, medium, high, xhigh, max (max = najgłębszy poziom dla modeli Anthropic)
`effortLevel (ustawienie)`	Trwały wysiłek rozumowania (domyślnie "medium")
`--context default|long_context`	Poziom okna kontekstu dla modeli z cennikiem warstwowym (contextTier)
`continueOnAutoMode`	Po limicie zapytań automatyczne przejście na Auto i ponowienie
`Ctrl+T`	Pokaż / ukryj rozumowanie modelu

```json
{ "model": "gpt-5.3-codex", "effortLevel": "low" }
```

Rozliczanie: AI Credits (stan: październik 2026)

Element	Opis
Model rozliczeń od 1.06.2026	Usage-based: koszt zależy od modelu i liczby tokenów (wejściowe, wyjściowe, z cache)
Jednostka	AI Credit — 1 AI Credit = 0,01 USD; Copilot CLI rozliczany w AI Credits
Copilot Pro (10 USD/mies.)	1 500 kredytów miesięcznie (1 000 bazowe + 500 flex)
Copilot Pro+ (39 USD/mies.)	7 000 kredytów (3 900 + 3 100)
Copilot Max (100 USD/mies.)	20 000 kredytów (10 000 + 10 000)
Copilot Free / Student	Pula kredytów, modele tylko przez Auto
Business / Enterprise	Kredyty w licencji, łączone na poziomie podmiotu rozliczeniowego; budżety: użytkownik, centrum kosztów, organizacja, enterprise
Auto	10% zniżki na koszt modeli w planach płatnych
Legacy: premium requests	Mnożniki per model — tylko dla rocznych planów Pro/Pro+ do końca okresu

Kontrola kosztów w CLI

Polecenie / opcja	Działanie
`/usage`	Statystyki sesji: tokeny per model, zużycie AI Credits per model (np. 1 AIC)
`/limits`	Okno limitów odpowiedzi
`/limits set max-ai-credits N`	Miękki limit kredytów (min. 30); bieżąca odpowiedź zawsze się kończy
`/limits unset [max-ai-credits|all]`	Usunięcie limitu
`--max-ai-credits=N`	Limit z wiersza poleceń (w -p przebieg kończy się po osiągnięciu limitu)
`/goal CEL --max-ai-credits N`	Budżet na cel autopilota
`/chronicle cost-tips`	Wskazówki oszczędzania na podstawie historii sesji
`copilot help billing`	Pomoc o rozliczeniach w terminalu
Stopka (/statusline)	Elementy showQuota i showAiUsed — limit i zużycie na bieżąco

BYOK — własny dostawca modeli

Zmienna / plik	Znaczenie
COPILOT_PROVIDER_BASE_URL (wymagana)	Adres API dostawcy
COPILOT_PROVIDER_TYPE	openai (domyślnie: OpenAI, Ollama, vLLM, Foundry Local, zgodne API), azure, anthropic
COPILOT_PROVIDER_API_KEY / COPILOT_PROVIDER_BEARER_TOKEN	Klucz API / token Bearer (token ma pierwszeństwo)
COPILOT_PROVIDER_API_KEY_COMMAND	Polecenie zwracające świeży klucz przed każdym żądaniem
COPILOT_PROVIDER_WIRE_MODEL · COPILOT_PROVIDER_MODEL_ID	Nazwa modelu wysyłana do dostawcy (Azure: deployment) · znany model (możliwości, limity)
COPILOT_PROVIDER_MAX_PROMPT_TOKENS · …_MAX_OUTPUT_TOKENS	Limity tokenów
COPILOT_PROVIDER_WIRE_API · COPILOT_PROVIDER_TRANSPORT	completions (domyślnie) lub responses (seria GPT-5) · http (domyślnie) lub websockets
COPILOT_PROVIDER_AZURE_API_VERSION · COPILOT_PROVIDER_HEADERS	Wersja API Azure (domyślnie trasa v1 bez wersji) · własne nagłówki Nazwa: wartość, jeden na linię
providers.json (COPILOT_PROVIDERS_CONFIG)	Rejestr wielu dostawców i modeli — gdy deklaruje dostawcę lub model, zmienne COPILOT_PROVIDER_* są pomijane
COPILOT_OFFLINE=true	Tryb offline: bez kontaktu z serwerami GitHub, bez telemetrii

```bash
export COPILOT_PROVIDER_BASE_URL=http://localhost:11434/v1   # np. lokalna Ollama
export COPILOT_MODEL=YOUR-MODEL-NAME
copilot
```

providers.json — format pliku

Pole	Znaczenie
`providers (tablica)`	Lista dostawców
`name · baseUrl`	Wymagane: unikalna nazwa dostawcy (bez znaku /) · adres API
`type · wireApi · transport`	openai (domyślnie), azure, anthropic · completions (domyślnie) lub responses · http lub websockets
`apiKey · bearerToken · headers`	Klucz API · token Bearer (wygrywa z kluczem) · dodatkowe nagłówki (obiekt)
`azure.apiVersion`	Wersja API Azure — bez niej CLI używa trasy /openai/v1
`models (tablica, opcjonalna)`	Modele przypisane do dostawców
`id · provider`	Wymagane: identyfikator modelu (unikalny w obrębie dostawcy) · nazwa dostawcy
`wireModel · modelId · name`	Nazwa wysyłana do API (domyślnie id) · znany model bazowy: zachowanie i limity (domyślnie id) · nazwa w selektorze
`maxPromptTokens · maxContextWindowTokens · maxOutputTokens`	Limity tokenów modelu

providers.json — format pliku (cd.)

```json
{
  "providers": [
    { "name": "lokalny", "type": "openai", "baseUrl": "http://localhost:11434/v1" },
    { "name": "azure-prod", "type": "azure", "baseUrl": "https://my-resource.openai.azure.com",
      "apiKey": "<klucz>", "azure": { "apiVersion": "2024-10-21" } }
  ],
  "models": [
    { "id": "coder", "provider": "lokalny", "wireModel": "qwen3-coder:30b",
      "name": "Qwen3 Coder (lokalnie)", "maxPromptTokens": 100000, "maxOutputTokens": 8192 },
    { "id": "gpt5", "provider": "azure-prod", "wireModel": "my-gpt5-deployment", "modelId": "gpt-5.4" }
  ]
}
```

```bash
copilot --model lokalny/coder        # identyfikator modelu: dostawca/id
```

providers.json — zasady działania (test CLI 1.0.91)

Sytuacja	Zachowanie
Format pliku	Ścisły JSON — bez komentarzy i przecinków na końcu (inaczej niż settings.json); providers i models muszą być tablicami
Nieznane pola	Ignorowane bez ostrzeżenia — literówka w nazwie pola nie daje błędu
Błędy blokujące konfigurację	Brak name, baseUrl, id lub provider; zły typ wartości; / w nazwie dostawcy; powtórzona nazwa dostawcy lub para dostawca/id; model z nieznanym dostawcą
Zmienne COPILOT_PROVIDER_*	Pomijane, gdy plik deklaruje dostawcę lub model (także klucz i adres); brak pliku lub pusty plik → CLI używa zmiennych
Sekrety	Wartości są dosłowne: ${ZMIENNA} nie jest rozwijane, pole apiKeyCommand nie działa — klucz w pliku to jawny tekst
Wybór modelu	Tylko pełny identyfikator dostawca/id; samo id kieruje do modeli GitHub; bez wyboru → „No supported model available”
Bez sekcji models	CLI pobiera listę z GET {baseUrl}/models i sam używa wykrytego modelu (test z jednym modelem); --model dostawca/… działa tylko dla modeli z models

Typ dostawcy → żądanie wysyłane przez CLI

Konfiguracja dostawcy	Żądanie HTTP	Klucz w nagłówku
`openai (domyślnie, wireApi: "completions")`	`POST {baseUrl}/chat/completions`	`Authorization: Bearer`
`openai + wireApi: "responses"`	`POST {baseUrl}/responses`	`Authorization: Bearer`
`azure bez azure.apiVersion`	`POST {baseUrl}/openai/v1/chat/completions`	`api-key`
`azure z azure.apiVersion`	`POST {baseUrl}/openai/deployments/{wireModel}/chat/completions?api-version=…`	`api-key`
`anthropic`	`POST {baseUrl}/v1/messages`	`x-api-key`

Zasady
Pole model w treści żądania = wireModel (domyślnie id)
bearerToken zawsze trafia do Authorization i ma pierwszeństwo przed apiKey
headers trafiają wyłącznie do dostawcy, nigdy do usług GitHub Copilot

Część 8

08

Kontekst, instrukcje i pamięć

Co agent wie o projekcie, zanim napisze pierwszą linię



Jak dostarczyć kontekst

Sposób	Działanie
`@ścieżka/do/pliku`	Zawartość pliku trafia do kontekstu promptu
`#NUMER`	Issue lub pull request z GitHub (treść, komentarze)
`--attachment ŚCIEŻKA`	Plik (także obraz) dołączony do pierwszego promptu, głównie w trybie -p; opcję można powtarzać
`Ctrl+V / Alt+V`	Wklejenie schowka / obrazu jako załącznika
Zakładki Issues/PR + c	Wstawienie odwołania do zaznaczonego elementu
`/add-dir, --add-dir`	Dodatkowy katalog (np. drugie repozytorium, biblioteka współdzielona)
Serwery MCP	Dane z systemów zewnętrznych: bazy, tickety, dokumentacja

Okno kontekstu: /context i /compact

`/context`	model, zajęte/całkowite tokeny oraz podział: System Prompt, Custom Instructions, System Tools, MCP Tools, Messages, Free Space, Buffer
Auto-kompaktowanie: od ok. 80% okna kompresja startuje w tle; przy ok. 95% CLI na chwilę wstrzymuje pracę do jej zakończenia	
`/compact [INSTRUKCJE]`	ręczne podsumowanie historii (Esc anuluje); po kompaktowaniu: podsumowanie + oryginalne instrukcje + bieżące plany i todo
Każde kompaktowanie tworzy punkt kontrolny (/session checkpoints); kompaktowania nie da się cofnąć	
Wyniki narzędzi > 20 KiB trafiają do pliku tymczasowego — model dostaje ścieżkę i podgląd (COPILOT_LARGE_OUTPUT_THRESHOLD_BYTES, domyślnie 20480)	
`toolSearch`	odroczone ładowanie narzędzi (mniej tokenów na definicje MCP)
`/clear (/new, /reset)`	czysty kontekst; /ask — pytanie bez dopisywania do historii

```text
/context
/compact skup się na decyzjach architektonicznych i otwartych TODO
```

Instrukcje niestandardowe — skąd są ładowane

Lokalizacja	Zakres
`AGENTS.md`	Katalog główny Git i katalog bieżący
`CLAUDE.md, GEMINI.md`	Katalog główny Git i katalog bieżący
`.github/copilot-instructions.md`	Katalog główny Git i katalog bieżący
`.github/instructions/**/*.instructions.md`	Katalog główny Git i katalog bieżący (instrukcje kontekstowe)
`.claude/rules/**/*.md`	Katalog główny Git i katalog bieżący (reguły Claude Code)
`$HOME/.copilot/copilot-instructions.md`	Osobiste — wszystkie projekty
`$HOME/.copilot/instructions/**/*.instructions.md`	Osobiste instrukcje kontekstowe
`COPILOT_CUSTOM_INSTRUCTIONS_DIRS`	Dodatkowe katalogi (lista rozdzielona przecinkami)

Instrukcje: zakres, importy, przełączanie

Instrukcje kontekstowe zawężamy kluczem applyTo lub paths we frontmatterze (glob ścieżek)	
Importy @ścieżka na początku linii wklejają treść innego pliku (rekurencyjnie, z limitami głębokości, cykli i rozmiaru) — w AGENTS.md, CLAUDE.md i .github/copilot-instructions.md	
`/instructions`	podgląd i włączanie/wyłączanie plików instrukcji w sesji
`--no-custom-instructions`	uruchomienie bez instrukcji z AGENTS.md i pokrewnych plików
`copilot instruction list`	lista źródeł bez otwierania sesji

```markdown
---
paths: "src/api/**/*.ts"
description: API layer conventions
---
Use the shared `ApiError` type for all thrown errors in this directory.
```

```markdown
# AGENTS.md
@docs/architecture.md
@docs/testing-guidelines.md
Build: `npm run build` · Test: `npm test` · Lint: `npm run lint`
```

/init i copilot init — instrukcje dla repozytorium

Copilot analizuje kod i tworzy lub aktualizuje .github/copilot-instructions.md
Typowa treść: polecenia build/test/lint, architektura wysokiego poziomu, konwencje specyficzne dla kodu
Gdy plik istnieje — agent proponuje ulepszenia do zaakceptowania lub odrzucenia
Przy braku instrukcji CLI wyświetla podpowiedź przy starcie; /init suppress wyłącza ją na stałe dla repozytorium
/init w sesji przygotowuje instrukcje i funkcje agentowe repozytorium
copilot init --sandbox / --no-sandbox — sandbox tylko dla sesji inicjalizacji **[EXP]**

Copilot Memory — pamięć agenta

Publiczny preview, wszystkie płatne plany; w Business/Enterprise najpierw polityka administratora **[ORG]**
Fakty na poziomie repozytorium (z cytatami kodu, weryfikowane względem bieżącej gałęzi; tworzone tylko z działań osób z prawem zapisu)
Preferencje na poziomie użytkownika — CLI stosuje fakty repozytorium i preferencje osoby, która uruchomiła sesję
Nieużywane wpisy są usuwane automatycznie po 28 dniach
Zapis nowych faktów to narzędzie memory — można je dopuścić (--allow-tool=memory) lub zablokować; korzystanie z istniejącej pamięci nie wymaga zgody
W trybie -p pamięć jest domyślnie wyłączona — --enable-memory ją włącza
Sterowanie w sesji wg GitHub Changelog (05.2026): /memory on, /memory off, /memory show

/chronicle — wiedza z historii sesji

Podkomenda	Działanie
`/chronicle standup`	Podsumowanie pracy (domyślnie 24 h; np. „for the last 3 days”)
`/chronicle tips`	3–5 spersonalizowanych wskazówek pracy z Copilotem
`/chronicle cost-tips`	Wskazówki dot. zużycia tokenów
`/chronicle search`	Pełnotekstowe wyszukiwanie w historii sesji
`/chronicle improve`	Propozycje ulepszeń .github/copilot-instructions.md dla bieżącego repozytorium
`/chronicle reindex`	Przebudowa lokalnego indeksu sesji i synchronizacja z kontem
`/chronicle skills create|review|status`	Propozycje skills repozytorium na podstawie obserwowanego użycia

Część 9

09

Sesje, równoległość i współpraca

Cykl życia sesji: wznawianie

Opcja / polecenie	Działanie
`--continue`	Wznawia ostatnią sesję z bieżącego katalogu (lub globalnie ostatnią)
`-r, --resume[=WARTOŚĆ]`	Selektor sesji lub wskazanie: ID, prefiks ID, nazwa sesji
`--session-id ID`	Dokładne ID sesji/zadania; nowa sesja tylko, gdy wartość to poprawny UUID
`-n, --name=NAZWA`	Nazwa nowej sesji (do wyszukania przez --resume i /resume)
`/resume [ID] (/continue)`	Przełączenie na inną sesję z listy
`/rename [NAZWA]`	Zmiana nazwy (bez argumentu — nazwa generowana automatycznie)
`/clear, /new, /reset [PROMPT]`	Nowa rozmowa (opcjonalnie od razu z promptem)

Selektor sesji	s — sortowanie (relevance → created → name → last used), Tab — lokalne/zdalne, x — usuń, Enter — otwórz

Zarządzanie sesjami

Polecenie	Działanie
`/session · /session info · /session id`	Informacje o sesji (ścieżka logu, ID, czas, katalog, link)
`/session checkpoints [n]`	Punkty kontrolne sesji
`/session files · /session plan`	Pliki sesji · plan sesji
`/session rename [NAZWA]`	Zmiana nazwy
`/session prune --older-than DNI [--dry-run] · /session cleanup`	Porządkowanie starych sesji
`/session delete [ID] [--yes] · /session delete-all --yes`	Usuwanie sesji
`/exit, /quit · /exit print`	Zamyka bieżącą sesję (CLI kończy się przy ostatniej) · zamyka CLI z propozycją zrzutu transkrypcji
`/restart`	Restart CLI z przywróceniem wszystkich aktywnych sesji

Wiele sesji naraz: pasek boczny

Obsługa (pusty prompt)	
←	otwiera pasek / przenosi fokus; → — wraca / zamyka
↑/↓ (k/j)	wybór sesji; Enter — przełączenie
n	nowa sesja w tle; x,x — zamknięcie
s	sortowanie: recent → created → name → none
Zakładka Sessions: a — filtr all/local/remote; / — wyszukiwanie (nazwa, gałąź, repo, ID)	
Mysz: klik, dwuklik, przeciąganie separatora	

Statusy sesji	
● zielona	bezczynna, czeka na prompt
● fioletowa pulsująca	agent pracuje
● niebieska	skończył turę, gdy byłeś gdzie indziej
● żółta	błąd lub wymaga uwagi
○ szara	zapisana, do wznowienia
! żółty	czeka na zgodę
? żółty	czeka na odpowiedź lub wybór

Rozgałęzianie sesji i Git worktree

Polecenie / opcja	Działanie
`/fork [NAZWA] (/branch)`	Kopia bieżącej sesji jako nowa sesja — działa także w trakcie pracy agenta
`/fork worktree`	Kopia sesji z kontekstem rozmowy w nowym worktree (zawsze od HEAD)
`-w, --worktree[=NAZWA]`	Start sesji w izolowanym worktree (domyślnie <repo>.worktrees/)
`/worktree [gałąź|zadanie]`	Nowy worktree i przełączenie; niezatwierdzone zmiany zostają w starym
`/new worktree [PROMPT]`	Pusta sesja w nowym worktree (zamiast przestarzałego /worktree new)
`/move [gałąź|zadanie]`	Przenosi niezatwierdzone zmiany do nowego worktree i przełącza
`worktreeBaseRef`	"head" (domyślnie) lub "defaultBranch" — punkt startowy nowych worktree
`worktreePathTemplate`	`Lokalizacja, np. ~/src/worktrees/{repo}/{branch}; symbole {repoPath}, {repo}, {branch}, {branchSlug}`

Udostępnianie i eksport sesji

Polecenie / opcja	Wynik
`/share (/export)`	Link GitHub do sesji (gdy zalogowany i zsynchronizowany), w przeciwnym razie eksport Markdown
`/share link · /share off`	Jawne udostępnienie linkiem · zatrzymanie udostępniania
`/share file [session|research] [ŚCIEŻKA]`	Eksport do Markdown
`/share html [session|research] [ŚCIEŻKA]`	Eksport do HTML
`/share gist [session|research]`	GitHub gist (niedostępne dla kont EMU i GHE.com)
`/share research [ŚCIEŻKA]`	Raport z /research
`--share=ŚCIEŻKA (tryb -p)`	Po zakończeniu: Markdown (domyślnie ./copilot-session-<ID>.md)
`--share-gist (tryb -p)`	Po zakończeniu: sekretny gist

Zdalne sterowanie sesją (GitHub.com i GitHub Mobile)

Polecenie / opcja	Działanie
`/remote · /remote on · /remote off`	Status (z kodem QR: Ctrl+O) · włączenie sterowania · zakończenie
`--remote / --no-remote`	Włącza / wyłącza zdalny dostęp dla tej sesji (ustawienie remoteSessions)
`--remote-export / --no-remote-export`	Eksport sesji tylko do podglądu / brak eksportu (wyłącza też sterowanie)
`--connect[=SESSION-ID]`	Bezpośrednie połączenie z sesją zdalną (ID sesji lub zadania)
`--remote --resume TASK-ID`	Wznowienie zdalnego zadania lokalnie

Co można zdalnie
podgląd, odpowiedzi na prośby o zgodę i pytania, akceptacja planów, nowe prompty, zmiana trybu, anulowanie. Czego nie: polecenia slash.

Integracja z VS Code: /ide

Automatyczne połączenie, gdy katalog roboczy CLI odpowiada folderowi otwartemu w VS Code w trybie zaufanym („Visual Studio Code connected”)
Działa z terminala zintegrowanego VS Code i z terminala zewnętrznego; jedno IDE naraz
Zaznaczenie w edytorze trafia do kontekstu promptu
Proponowane zmiany jako diff obok siebie w VS Code — akceptacja ✓ lub odrzucenie ✗
Diagnostyka na żywo; sesje CLI widoczne w VS Code (Chat → Sessions → „Resume in Terminal”)
/ide — status, połączenie z innym workspace, rozłączenie; ustawienia ide.autoConnect, ide.openDiffOnEdit
Ograniczenia: brak podglądu diffów przy uprawnieniach „allow-all”; lokalne CLI nie połączy się z VS Code w zdalnym Codespace

Aplikacja GitHub Copilot, Windows i chmura

Element	Działanie
`copilot app`	Otwiera aplikację GitHub Copilot w bieżącym katalogu (nowa sesja przez deep link) lub stronę pobierania
`/app`	Otwiera bieżącą sesję w aplikacji (wymaga aplikacji 1.1.3+)
Wspólne sesje	Aplikacja i CLI przechowują sesje w ~/.copilot/session-state/
`taskbarPresence`	Windows 11: sesje Copilot na pasku zadań z podglądem statusu
`terminalNotifications` · `terminalProgress`	Powiadomienia OSC 777 (Ghostty, WezTerm) · wskaźnik postępu OSC 9;4 (Windows Terminal, iTerm2…)
`copilot --cloud --experimental` **[EXP]**	Interaktywna sesja CLI w efemerycznym sandboksie Linux w chmurze GitHub (nie łączy się z -p/-i; polityka „Cloud Sandbox access” **[ORG]**)

Harmonogramy w sesji [EXP]

`/every [INTERWAŁ] PROMPT`	cykliczne uruchamianie promptu, skilla lub polecenia slash
`/after [OPÓŹNIENIE] PROMPT`	jednorazowe uruchomienie po czasie
Samo /every lub /after — menedżer harmonogramów (↑/↓ wybór, x usunięcie; dodawanie tylko z promptu)	
Stopka pokazuje „N scheduled”, gdy sesja ma aktywne harmonogramy	

```text
/every 1h uruchom testy i podsumuj błędy
/every 1d /chronicle standup
/after 30m przypomnij mi o code review
```

Część 10

10

Serwery MCP

Podłączanie agenta do narzędzi i danych spoza repozytorium

MCP w Copilot CLI — podstawy

- MCP (Model Context Protocol) — otwarty standard udostępniania agentowi narzędzi i danych z systemów zewnętrznych (bazy, tickety, dokumentacja, przeglądarka…)
- Wbudowany serwer GitHub MCP (github-mcp-server) — gotowy bez konfiguracji, domyślnie z narzędziami tylko do odczytu; podgląd: /mcp show github-mcp-server
- Gdzie definiujemy serwery:
  - ~/.copilot/mcp-config.json — serwery użytkownika (trwałe)
  - .mcp.json (od katalogu bieżącego do korzenia repozytorium) lub .github/mcp.json — serwery projektu, ładowane dopiero w zaufanym folderze
  - wtyczki, agenci (mcp-servers we frontmatterze) i --additional-mcp-config (jedna sesja)
- .vscode/mcp.json z VS Code NIE jest czytany przez CLI (inny klucz główny: servers)
- Uprawnienia do narzędzi MCP: reguły NAZWA-SERWERA i NAZWA-SERWERA(narzędzie)

Opcje MCP wiersza poleceń

Opcja / zmienna	Działanie
`--additional-mcp-config=JSON lub =@plik.json`	Serwery tylko dla tej sesji; uzupełnia mcp-config.json, nadpisuje serwer o tej samej nazwie
`--disable-mcp-server=NAZWA`	Wyłącza serwer (opcję można powtarzać)
`--enable-mcp-server=NAZWA`	Włącza na tę sesję serwer wyłączony w ustawieniach (bez zapisu)
`--disable-builtin-mcps`	Wyłącza wbudowane serwery (obecnie github-mcp-server)
`--add-github-mcp-tool=NARZĘDZIE`	Dodaje narzędzie GitHub MCP ponad domyślny podzbiór (* = wszystkie)
`--add-github-mcp-toolset=ZESTAW`	Dodaje zestaw narzędzi GitHub MCP (all = wszystkie)
`--enable-all-github-mcp-tools`	Wszystkie narzędzia GitHub MCP (nadpisuje dwie powyższe opcje)
`--allow-all-mcp-server-instructions`	Instrukcje inicjalizacyjne wszystkich serwerów od razu w prompcie systemowym (domyślnie tylko z listy dozwolonych, reszta na żądanie)

Skąd pochodzą serwery MCP — pierwszeństwo i zaufanie

Zasada	Szczegóły
Pliki projektu	CLI przechodzi od katalogu bieżącego do korzenia repozytorium; bliższy plik wygrywa przy tej samej nazwie
.mcp.json vs .github/mcp.json	W tym samym katalogu wygrywa .mcp.json
Projekt vs użytkownik	Definicje projektu wygrywają z ~/.copilot/mcp-config.json
Użytkownik → wtyczki → sesja	Ostatni wygrywa: mcp-config.json → konfiguracje wtyczek → --additional-mcp-config
Zaufanie	Serwery projektu ładują się dopiero po zaufaniu folderu; w niezaufanym są pomijane bez komunikatu
Tryb -p	Serwery projektu tylko w zaufanym folderze lub z GITHUB_COPILOT_PROMPT_MODE_WORKSPACE_MCP=true
Polityka firmy	allowedMcpServers / deniedMcpServers, adres własnego rejestru MCP i allowlista organizacji

Serwer lokalny (stdio) — format konfiguracji

Pole	Wymagane	Opis
`command · args`	tak	Polecenie uruchamiające serwer i jego argumenty (tablica)
`tools`	tak	["*"] lub lista nazw narzędzi
`env`	nie	Zmienne środowiskowe ($VAR, ${VAR}, ${VAR:-domyślna}); automatycznie dziedziczony jest tylko PATH
`cwd`	nie	Katalog roboczy serwera
`timeout`	nie	Limit (ms) wykrywania i wywołań narzędzi; domyślnie 30000
`type`	nie	"local" (domyślnie) lub "stdio"
`deferTools`	nie	"auto" (domyślnie) lub "never" — narzędzia zawsze widoczne mimo wyszukiwania narzędzi
`disableToolCache`	nie	true = bez migawki listy narzędzi
`slowConnectionThresholdMs`	nie	Po ilu ms ostrzec o wolnym połączeniu (domyślnie 10000; tylko ostrzeżenie)

```json
{
  "mcpServers": {
    "playwright": { "type": "local", "command": "npx", "args": ["@playwright/mcp@latest"], "tools": ["*"] },
    "my-internal-server": {
      "command": "npx",
      "args": ["--registry", "https://npm.pkg.github.com", "@my-org/internal-mcp-server"],
      "tools": ["*"]
    }
  }
}
```

Serwer zdalny (HTTP/SSE) i OAuth

Pole	Opis
`type · url · tools`	"http" lub "sse" ("streamable-http" = alias http) · adres serwera · narzędzia (wymagane)
`headers`	Nagłówki HTTP (z rozwijaniem zmiennych)
`oauthClientId`	Statyczny client ID (pomija dynamiczną rejestrację)
`oauthScopes`	Lista zakresów OAuth (wymaga oauthClientId)
`oauthPublicClient`	Klient publiczny (domyślnie true); false dla klienta poufnego z sekretem
`oauthGrantType`	"authorization_code" (domyślnie, przeglądarka) lub "client_credentials" (bez przeglądarki)
`oidc`	true = wstrzyknięcie tokenu OIDC (nagłówek Bearer dla zdalnych; GITHUB_COPILOT_OIDC_MCP_TOKEN[_SUFIKS] w env dla lokalnych)
`timeout · deferTools`	Jak dla serwerów lokalnych

json · przykład poglądowy

```
{
  "mcpServers": {
    "tickets": {
      "type": "http",
      "url": "https://mcp.example.com/mcp",
      "headers": { "Authorization": "Bearer ${TICKETS_TOKEN}" },
      "tools": ["*"]
    }
  }
}
```

copilot mcp — zarządzanie bez sesji

Podkomenda	Działanie
`list [--json]`	Wszystkie serwery wg źródła (użytkownik, projekt, wtyczki); wyłączone oznaczone (disabled)
`get <nazwa> [--json]`	Konfiguracja, status i narzędzia; dla wtyczek — nazwa i wersja wtyczki
`add [opcje] <nazwa> [url]`	Dodaje serwer do ~/.copilot/mcp-config.json
`enable <nazwa> · disable <nazwa>`	Trwałe włączenie / wyłączenie (przyszłe sesje)
`remove <nazwa>`	Usuwa serwer użytkownika (serwery z plików projektu edytuje się w pliku)

Opcje add	-- polecenie [arg...] (stdio), <url>, --transport stdio|http|sse (domyślnie stdio), --env KLUCZ=WARTOŚĆ, --header "Nagłówek: wartość", --tools "*"|lista|"", --timeout <ms>, --json, --show-secrets

```bash
copilot mcp add context7 -- npx -y @upstash/context7-mcp
copilot mcp add --transport http notion https://mcp.notion.com/mcp
copilot mcp add --transport http tickets https://mcp.example.com/mcp \
  --header "Authorization: Bearer $TICKETS_TOKEN" --tools "*"
copilot mcp list --json
copilot mcp disable tickets
```

/mcp w sesji interaktywnej

Polecenie	Działanie
`/mcp` · `/mcp config`	Panel wtyczek otwarty na liście serwerów MCP
`/mcp show [NAZWA]`	Szczegóły serwera, lista narzędzi, włączanie/wyłączanie, źródło (np. wtyczka)
`/mcp list (ls)`	Lista tekstowa ze statusem połączenia
`/mcp add` · `/mcp edit NAZWA`	Formularz dodawania / edycji (Tab między polami, Ctrl+S zapisuje)
`/mcp delete NAZWA`	Usunięcie serwera
`/mcp enable` · `/mcp disable NAZWA`	Włączenie / wyłączenie
`/mcp auth NAZWA`	Ponowne uwierzytelnienie (OAuth)
`/mcp reload`	Przeładowanie serwerów
`/mcp search [ZAPYTANIE]` **[EXP]**	Wyszukiwanie w GitHub MCP Registry (lub rejestrze organizacji) i instalacja z gotowego formularza

Część 11

11

Skills, agenci, hooki, wtyczki i LSP

Zamiana wiedzy i zasad zespołu w powtarzalne mechanizmy



Mechanizmy rozszerzeń — kiedy czego użyć

Mechanizm	Kiedy działa	Do czego służy	Przykład
Instrukcje (AGENTS.md…)	Zawsze, w każdej turze	Konwencje, polecenia build/test, zasady	„Testy uruchamiaj przez npm test”
Skills	Na żądanie — gdy pasuje opis lub /nazwa-skilla	Procedury krok po kroku ze skryptami i zasobami	Skill „release-notes”
Agenci niestandardowi	Gdy wybrani lub dobrani po opisie; własny kontekst	Wyspecjalizowana rola z własnymi narzędziami i modelem	`security-auditor`
Hooki	Deterministycznie, w zdarzeniach cyklu życia	Egzekwowanie zasad, audyt, automatyzacja	preToolUse blokujący rm -rf
Serwery MCP	Jako narzędzia w sesji	Dostęp do systemów zewnętrznych	Tickety, baza danych, przeglądarka
LSP	W tle	Inteligencja kodu: definicje, odwołania, diagnostyka	`typescript-language-server`
Wtyczki	Po instalacji	Dystrybucja wszystkich powyższych w jednym pakiecie	Wtyczka standardów zespołu

Skills — wiedza ładowana na żądanie

Skill = folder z plikiem SKILL.md (frontmatter YAML + instrukcje) oraz opcjonalnymi skryptami i zasobami
Otwarty standard Agent Skills — te same skills działają w CLI, Copilot cloud agent, code review, aplikacji Copilot i trybie agenta w VS Code/JetBrains
Użycie automatyczne: Copilot dobiera skill po description i wstrzykuje SKILL.md do kontekstu
Użycie jawne: /nazwa-skilla w prompcie, np. „Use the /frontend-design skill to…”
Frontmatter: name (wymagane; małe litery i myślniki), description (wymagane: co robi i kiedy), license, allowed-tools (narzędzia bez pytania o zgodę), argument-hint, user-invocable, disable-model-invocation

```markdown
---
name: image-convert
description: Converts SVG images to PNG format. Use when asked to convert SVG files.
allowed-tools: shell
---
When asked to convert an SVG to PNG, run the `convert-svg-to-png.sh` script
from this skill's base directory, passing the input SVG file path as the first argument.
```

Skills — lokalizacje i zarządzanie

Element	Szczegóły
Projekt	.github/skills/, .agents/skills/, .claude/skills/ (także w katalogach nadrzędnych)
Osobiste	`~/.copilot/skills/, ~/.agents/skills/`
Dodatkowe	`COPILOT_SKILLS_DIRS, ustawienie skillDirectories, /skills add KATALOG; wtyczki: skills/`
Deduplikacja	Po polu name — pierwszy załadowany wygrywa (projekt przed osobistymi i wtyczkami)
`/skills`	Panel: włączanie/wyłączanie (spacja/Enter)
`/skills list · info NAZWA · reload`	Lista · szczegóły (lokalizacja, wtyczka) · przeładowanie
`/skills add [--project] PLIK|URL|KATALOG · /skills remove NAZWA|KATALOG`	Dodanie (osobiste lub projektu) · usunięcie
`copilot skill list [--json] · add [--project] · remove · enable · disable`	To samo bez sesji
Ustawienia	`disabledSkills, ignoredSkillsLocations, dynamicRetrieval.skills`

Agenci niestandardowi (.agent.md)

Profil agenta = plik Markdown z frontmatterem YAML: opis, narzędzia, model, serwery MCP + prompt (do 30 000 znaków)
Praca agenta odbywa się w subagencie z własnym oknem kontekstu
Tworzenie: /agent → Create new agent → Project (.github/agents/) lub User (~/.copilot/agents/) → „Use Copilot” (opis słowny) lub ręcznie; po utworzeniu zrestartuj CLI
Użycie: /agent (wybór), polecenie w prompcie („użyj agenta security-auditor…”), automatyczny dobór po description, --agent NAZWA (także z -p), @NAZWA w promptach /fleet

```markdown
---
name: security-auditor
description: Audits code changes for OWASP Top 10 vulnerabilities
tools: ["read", "search", "execute"]
model: claude-opus-5.5
include-custom-instructions: true
---
You are a security specialist. Review the changes and report only
high-confidence, exploitable issues with clear remediation steps.
```

```bash
copilot --agent security-auditor -p "Przejrzyj zmiany w src/auth" --allow-tool='shell(git:*)'
```

Frontmatter agenta — wszystkie pola

Pole	Znaczenie
`name`	Nazwa wyświetlana (opcjonalna; domyślnie z nazwy pliku)
`description`	Wymagane — cel i możliwości; na tej podstawie agent jest dobierany automatycznie
`tools`	Lista lub ciąg rozdzielony przecinkami; brak lub ["*"] = wszystkie, [] = żadne; MCP: serwer/narzędzie, serwer/*
`model`	Model agenta (domyślnie dziedziczony); można podać kilka — próbowane po kolei
`model-policy`	required — zmiana modelu tylko w obrębie listy z pola model; preferred — model można nadpisać
`disable-model-invocation`	true = tylko ręczny wybór (zastępuje wycofane infer: false)
`user-invocable`	false = tylko wywołanie programowe (ukryty w /agent)
`include-custom-instructions`	CLI: subagent stosuje instrukcje repozytorium (AGENTS.md, CLAUDE.md…)
`mcp-servers`	Dodatkowe serwery MCP agenta (format jak mcp-config.json, w YAML)
`target · metadata`	vscode lub github-copilot (domyślnie oba) · dowolne adnotacje

Aliasy narzędzi	`execute (shell), read, edit, search, agent, web, todo`

Agenci — lokalizacje i kolejność ładowania

Kolejność	Lokalizacja	Zakres
1	`~/.copilot/agents/`	Użytkownik — wszystkie sesje
2–3	.github/agents/ w projekcie i katalogach nadrzędnych	Repozytorium / monorepo
4–5	.claude/agents/ w projekcie i katalogach nadrzędnych	Zgodność z Claude Code
6	.github/agents/ katalogów z --add-dir	Dodane katalogi (zaufane)
7	agents/ zainstalowanych wtyczek	Wtyczki (wg kolejności instalacji)
8	agents/ w repozytorium .github lub .github-private organizacji/enterprise	Agenci firmowi (zdalnie)

Zasady
Pierwszy załadowany wygrywa (deduplikacja po ID); ID = nazwa pliku bez .agent.md (podkatalogi → --, np. security--security-auditor)
Agenci wbudowani są zawsze obecni i nie da się ich nadpisać
customAgents.defaultLocalOnly: true — bez agentów organizacji i enterprise

Hooki — zdarzenia sesji i promptów

Zdarzenie	Kiedy	Wpływ na przebieg
`sessionStart`	Start nowej lub wznowionej sesji	additionalContext; hook typu prompt może wysłać prompt lub polecenie slash
`sessionEnd`	Koniec sesji (complete, error, abort, timeout, user_exit)	Tylko powiadomienie
`userPromptSubmitted`	Użytkownik wysyła prompt	Modyfikacja promptu tylko w hookach SDK
`userPromptTransformed`	Po przekształceniu promptu w treść dla modelu	Może podmienić treść dla modelu
`preCompact`	Przed kompaktowaniem kontekstu (manual/auto)	Tylko powiadomienie
`notification`	Powiadomienia systemowe (zakończenie shella/agenta, prośba o zgodę…)	Asynchronicznie; opcjonalnie additionalContext
`errorOccurred`	Błąd (model, narzędzie, system, wejście)	Tylko powiadomienie

Hooki — zdarzenia narzędzi i agentów

Zdarzenie	Kiedy	Wpływ na przebieg
`preToolUse`	Przed każdym użyciem narzędzia	allow / deny / ask, podmiana argumentów (modifiedArgs)
`permissionRequest`	Przed standardowym sprawdzeniem uprawnień	Programowe allow / deny (opcjonalnie zatrzymanie agenta)
`postToolUse`	Po udanym użyciu narzędzia	Podmiana wyniku, dodatkowy kontekst
`postToolUseFailure`	Po nieudanym użyciu narzędzia	Wskazówki naprawcze dla agenta
`agentStop`	Agent główny kończy turę	block wymusza kolejną turę z podanym powodem
`subagentStart`	Start subagenta	Kontekst doklejany do promptu subagenta
`subagentStop`	Subagent kończy pracę	block lub podmiana odpowiedzi (modifiedResponse)

Hooki — gdzie leżą i jakie mają typy

Źródło (kolejność ładowania)	Lokalizacja
Polityka administratora	/etc/github-copilot/policy.d/*.json, C:\ProgramData\GitHub\Copilot\policy.d*.json lub rejestr — zawsze aktywne, zawsze poza sandboksem
Użytkownik	~/.copilot/hooks/*.json lub klucz hooks w ~/.copilot/settings.json
Repozytorium	.github/hooks/*.json lub klucz hooks w .github/copilot/settings(.local).json; także .claude/settings.json
Wtyczki	hooks.json w pakiecie wtyczki
Programowo	SDK i rozszerzenia (hooki sesyjne)

Typy wpisów
`command`
bash / powershell / command (wieloplatformowo) lub exec + args (bez shella); cwd, env, timeoutSec (domyślnie 30)
`http`
POST z danymi zdarzenia na url (HTTPS wymagane dla preToolUse i permissionRequest); headers, allowedEnvVars
`prompt`
tylko sessionStart: automatycznie wysyła prompt lub polecenie slash w nowej sesji interaktywnej

Hook w praktyce: strażnik poleceń

```json
{
  "version": 1,
  "hooks": {
    "preToolUse": [
      { "type": "command", "matcher": "bash|powershell",
        "bash": "./.github/hooks/guard.sh", "timeoutSec": 10 }
    ],
    "postToolUse": [
      { "type": "http", "url": "https://hooks.example.com/copilot", "timeoutSec": 30 }
    ]
  }
}
```

```bash
#!/usr/bin/env bash
# .github/hooks/guard.sh — dane zdarzenia (JSON) przychodzą na stdin
input=$(cat)
if echo "$input" | grep -q 'rm -rf'; then
  echo '{"permissionDecision":"deny","permissionDecisionReason":"Polityka zespołu: zakaz rm -rf"}'
fi
# brak wyjścia = domyślny przepływ uprawnień
```

Hooki — decyzje, kody wyjścia, wyłączanie

Element	Zasada
preToolUse → wynik	permissionDecision: allow/deny/ask; permissionDecisionReason (wymagany przy deny); modifiedArgs
postToolUse → wynik	`modifiedResult, additionalContext (do 10 KB)`
agentStop / subagentStop → wynik	decision: "block" + reason = kolejna tura; modifiedResponse (tylko subagent)
permissionRequest → wynik	behavior: allow/deny, message, interrupt: true (zatrzymuje agenta)
Kod wyjścia 0	Sukces — stdout czytany jako JSON
Kod wyjścia 2	preToolUse i permissionRequest: odmowa; postToolUseFailure: kontekst dla agenta; inne: ostrzeżenie
Inny kod / błąd	Praca trwa dalej — wyjątek: błąd preToolUse = odmowa (fail-closed)
Przekroczenie czasu	Zawsze praca trwa dalej (fail-open), także dla preToolUse
Wyłączanie	"disableAllHooks": true w pliku, w ustawieniach repozytorium lub użytkownika — hooki polityki działają zawsze

Wtyczki (plugins) — pakiety rozszerzeń

Instalowalny pakiet: agenci, skills, hooki, konfiguracje serwerów MCP i LSP (także polecenia, reguły, rozszerzenia)
Działają w Copilot CLI, Copilot cloud agent i aplikacji GitHub Copilot
Dwa formaty: Agent Plugins 1.0 (otwarta specyfikacja, $schema w plugin.json — zalecany dla nowych) oraz format klasyczny (konfigurowalne ścieżki komponentów)
Zmienne: ${PLUGIN_ROOT} (katalog wtyczki), ${PLUGIN_DATA} (trwały katalog danych w ~/.copilot/plugin-data/)

```text
my-plugin/                    # Agent Plugins 1.0
├── plugin.json               # manifest (wymagany)
├── skills/deploy/SKILL.md
├── mcp.json                  # serwery MCP (opcjonalnie)
└── com.github.copilot/
    ├── agents/helper.agent.md
    ├── hooks/hooks.json
    └── lsp.json
```

```json
{
  "$schema": "https://agent-plugins.org/schemas/1.0.0/plugin.schema.json",
  "name": "my-dev-tools",
  "description": "React development utilities",
  "version": "1.2.0",
  "license": "MIT"
}
```

copilot plugin i /plugin

Polecenie	Działanie
`copilot plugin install SPEC (add)`	Instalacja: plugin@marketplace, OWNER/REPO, OWNER/REPO:ŚCIEŻKA, URL Git, ścieżka lokalna
`copilot plugin list [--json] · update NAZWA (--all)`	Lista · aktualizacja
`copilot plugin enable · disable NAZWA`	Włączenie / wyłączenie bez odinstalowania
`copilot plugin uninstall NAZWA (remove, rm)`	Odinstalowanie
`copilot plugin marketplace add SPEC · list · browse NAZWA`	Rejestracja i przeglądanie marketplace'ów
`copilot plugin marketplace update [NAZWA] · remove NAZWA [--force]`	Odświeżenie katalogu · wyrejestrowanie
`/plugin`	Panel: Installed / Online / Marketplace (←/→), akcje Enter, R odświeża katalogi
`--plugin-dir=KATALOG`	Wtyczka z katalogu lokalnego tylko dla tej sesji

```bash
copilot plugin install database-data-management@awesome-copilot
copilot plugin marketplace add anthropics/claude-code
copilot plugin install ./my-plugin          # rozwój lokalny (po zmianach — reinstalacja)
```

Marketplace i deklaratywne włączanie wtyczek

Element	Opis
`marketplace.json`	Jedyny wymagany plik marketplace'u (np. .github/plugin/marketplace.json); pola: name, owner, plugins (wymagane), metadata
Wpis wtyczki	name, source (ścieżka względna, GitHub z ref/sha, URL), description, version i inne
`enabledPlugins`	Deklaratywna instalacja: "plugin@marketplace": true/false (w ustawieniach użytkownika lub repozytorium)
`extraKnownMarketplaces`	Dodatkowe marketplace'y (źródło directory, git lub github)
`strictKnownMarketplaces (polityka)`	Allowlista marketplace'ów; [] blokuje wszystkie, także wbudowane

```json
{
  "name": "my-marketplace",
  "owner": { "name": "Your Organization", "email": "plugins@example.com" },
  "metadata": { "description": "Curated plugins for our team", "version": "1.0.0" },
  "plugins": [
    { "name": "security-checks", "description": "Check for potential security vulnerabilities",
      "version": "1.3.0", "source": "./plugins/security-checks" }
  ]
}
```

Serwery językowe (LSP)

Element	Szczegóły
Co dają	Przejście do definicji, odwołania, hover, zmiana nazwy, symbole dokumentu i przestrzeni roboczej, implementacje, wywołania przychodzące/wychodzące
Instalacja	Serwery nie są dołączone do CLI — zainstaluj je sam (np. npm install -g typescript typescript-language-server)
Konfiguracja (od najwyższego priorytetu)	.github/lsp.json (projekt) → wtyczki → ~/.copilot/lsp-config.json (użytkownik)
Pola	`command i fileExtensions (wymagane), args, env, rootUri, initializationOptions, requestTimeoutMs (90 s), initializationTimeoutMs (60 s), disabled`
W sesji	`/lsp (show), /lsp test NAZWA, /lsp reload, /lsp logs, /lsp help`
Bez sesji	`copilot lsp list [--json]`

```json
{
  "lspServers": {
    "typescript": {
      "command": "typescript-language-server", "args": ["--stdio"],
      "fileExtensions": { ".ts": "typescript", ".tsx": "typescriptreact", ".js": "javascript" }
    }
  }
}
```

Rozszerzenia CLI [EXP]

Rozszerzenie = moduł Node.js uruchamiany jako osobny proces obok sesji; dodaje narzędzia (dla agenta), polecenia slash (dla Ciebie) i hooki
Plik wejściowy extension.mjs (także .cjs/.js) w .github/extensions/NAZWA/, ~/.copilot/extensions/NAZWA/ lub we wtyczce
/extensions manage — menedżer (status, logi); /extensions mode — Load & Augment (domyślnie; Copilot może tworzyć i przeładowywać rozszerzenia), Load Only, Disabled
Rozszerzenia działają z Twoimi uprawnieniami; CLI usuwa z ich środowiska wrażliwe zmienne (np. GITHUB_TOKEN) — trzeba o nie poprosić
Rozszerzenia projektu ładują się tylko w zaufanym folderze; w -p: GITHUB_COPILOT_PROMPT_MODE_EXTENSIONS=true

```javascript
// .github/extensions/tool-time/extension.mjs
import { joinSession } from "@github/copilot-sdk/extension";

const session = await joinSession({
  tools: [{
    name: "session_tool_time",
    description: "Report how long tool calls have taken",
    parameters: { type: "object", properties: {} },
    handler: async () => "So far this session, ...",
  }],
  commands: [{ name: "tokencount", description: "Show token usage", handler: async (ctx) => {} }],
});
```

Dynamiczne workflowy [EXP]

Element	Szczegóły
Czym są	Program opisujący przebieg zadania: kroki automatyczne + agenci (sekwencyjnie lub równolegle), definiowany w rozszerzeniu
Status	Publiczny preview; w CLI wymaga trybu eksperymentalnego; plany z rozliczaniem tokenowym
Tworzenie	Poproś Copilota: „Create a dynamic workflow named review-changed that…" — zapis w rozszerzeniu sesji; udostępnianie przez ~/.copilot/extensions/, .github/extensions/ lub wtyczkę
Uruchamianie	Promptem (np. „Run the java-security-checks dynamic workflow… with an AI credit limit of 500") lub copilot workflow run NAZWA
`/workflows`	Przebiegi, fazy, agenci, kredyty; P pauza, X anulowanie, R wznowienie
Limity	`workflows.defaultLimits.maxConcurrentSubagents, maxTotalSubagents, timeoutSeconds, maxAiCredits`

Część 12

12

Konfiguracja i ustawienia

Katalog ~/.copilot, poziomy ustawień, polityki firmowe i diagnostyka

Katalog ~/.copilot — konfiguracja

Element	Zawartość
`settings.json`	Ustawienia użytkownika (JSON z komentarzami)
`config.json`	Stan aplikacji zarządzany automatycznie (logowanie, zainstalowane wtyczki, trustedFolders)
`mcp-config.json`	Serwery MCP użytkownika (.mcp.json / .github/mcp.json projektu wygrywają przy tej samej nazwie)
`lsp-config.json`	Serwery językowe użytkownika
`providers.json`	Rejestr dostawców i modeli BYOK
`permissions-config.json`	Zapamiętane zgody na narzędzia i katalogi per projekt
`copilot-instructions.md, instructions/`	Osobiste instrukcje
`agents/, skills/, hooks/, extensions/`	Osobiste agenci, skills, hooki, rozszerzenia

Katalog ~/.copilot — dane, stan i lokalizacje

Element	Zawartość
`session-state/`	Historia sesji: katalog per ID z events.jsonl, planami, punktami kontrolnymi, raportami
`session-store.db`	Baza SQLite do wyszukiwania między sesjami (/chronicle reindex)
`command-history-state/`	Historia poleceń (Ctrl+R)
`logs/`	Logi: process-{timestamp}-{pid}.log, logi rozszerzeń w logs/extensions/
`installed-plugins/, plugin-data/`	Pliki i dane zainstalowanych wtyczek
`mcp-oauth-config/, mcp-secrets/`	Awaryjny magazyn tokenów OAuth i sekretów MCP
`ide/`	Stan połączenia z IDE
`COPILOT_HOME`	Zastępuje cały katalog ~/.copilot (bez automatycznego przenoszenia danych)
Cache (COPILOT_CACHE_HOME)	macOS ~/Library/Caches/copilot, Linux ~/.cache/copilot, Windows %LOCALAPPDATA%/copilot

Kaskada ustawień — kto wygrywa

Poziom (od najsłabszego)	Źródło
1. Wartości wbudowane	Domyślne CLI
2. Ustawienia zarządzane MDM	Polityka urządzenia (baseline)
3. Użytkownik	`~/.copilot/settings.json`
4. Repozytorium	.github/copilot/settings.json (commitowane)
5. Lokalne	.github/copilot/settings.local.json (dodaj do .gitignore)
6. Zmienne środowiskowe	`np. COPILOT_MODEL, COPILOT_ALLOW_ALL`
7. Opcje wiersza poleceń	`np. --model, --allow-tool`

Wyjątki
permissions.disableBypassPermissionsMode: "disable" z MDM zawsze wygrywa
Zarządzany sandbox to „podłoga” — Twoje łagodniejsze ustawienie przegrywa
model, effortLevel, contextTier z repozytorium działają tylko w zaufanym katalogu

Jak zmieniać ustawienia

Sposób	Działanie
`/settings (/config)`	Okno ustawień z zakładkami User, Repo, Repo (local), Problems
`/settings KLUCZ` · `/settings KLUCZ WARTOŚĆ`	Okno ustawione na kluczu · zmiana wartości wprost
`/settings show KLUCZ`	Bieżąca wartość (wartości sekretne są maskowane)
`/settings --repo …` · `/settings --local …`	Zapis do ustawień repozytorium / lokalnych (tylko klucze dopuszczone w repo)
`Ctrl+E w /settings lub /sandbox`	Zapisuje zmiany i otwiera settings.json w edytorze
`copilot config … (1.0.92)`	Listowanie, odczyt, ustawianie i usuwanie ustawień z shella
Edycja pliku	settings.json (JSONC); błędne klucze trafiają do zakładki Problems

```text
/settings --repo model claude-sonnet-4.6
/settings stayInAutopilot false
/settings show model
```

Ustawienia repozytorium i lista dozwolonych modeli

Klucz w .github/copilot/settings.json	Łączenie z ustawieniami użytkownika
`model`, `effortLevel`, `contextTier`	Repozytorium wygrywa (tylko w zaufanym katalogu)
`companyAnnouncements`, `includeCoAuthoredBy`, `mergeStrategy`, `disableAllHooks`	Repozytorium wygrywa
`deniedUrls`, `disabledMcpServers`, `disabledSkills`	Suma — repozytorium może tylko dodać
`enabledPlugins`, `extraKnownMarketplaces`, `hooks`	Scalane — repozytorium wygrywa dla tego samego klucza
`respectGitignore`	Tylko zaostrzenie — repozytorium może włączyć, nie wyłączyć

```text
# .github/allowed_models.txt
fallback: gpt-6-astra
gpt-6-astra
gpt-5.4
claude-sonnet-*
```

Kluczowe ustawienia: model i zachowanie agenta

Klucz	Znaczenie (domyślnie)
`model` · `effortLevel` · `autoTier`	Model ("auto") · wysiłek rozumowania ("medium") · poziom Auto
`askUser`	Czy agent może zadawać pytania (true; false = autonomia)
`defaultMode` · `defaultPermissionMode`	Tryb startowy i tryb zatwierdzania nowych sesji interaktywnych
`stayInAutopilot`	Pozostanie w autopilocie po zakończeniu zadania (true)
`continueOnAutoMode`	Przejście na Auto po limicie zapytań (false)
`subagents.agents` · `subagents.disabledSubagents`	Modele per subagent · wyłączeni subagenci
`builtInAgents.rubberDuck` · `…rubberDuckAutoInvoke`	Rubber duck (true) · proaktywne wywołania (false)
`customAgents.defaultLocalOnly`	Tylko lokalni agenci, bez agentów organizacji (false)
`toolSearch` · `dynamicRetrieval`	Odroczone ładowanie narzędzi · dynamiczne pobieranie instrukcji/skills
`includeCoAuthoredBy` · `mergeStrategy`	Trailer Co-authored-by w commitach (true) · rebase/merge dla /pr fix conflicts

Kluczowe ustawienia: bezpieczeństwo, sieć, rozszerzenia

Klucz	Znaczenie
`allowedUrls` · `deniedUrls`	Adresy dozwolone bez pytania · zawsze blokowane (odmowa wygrywa)
`permissions.disableBypassPermissionsMode`	"disable" lub "allow-auto-only" — blokada trybów allow-all
`disableAllHooks` · `hooks`	Wyłączenie wszystkich hooków · hooki użytkownika w pliku ustawień
`disabledMcpServers` · `enabledMcpServers`	Wyłączone serwery MCP · włączone wbudowane serwery domyślnie wyłączone
`disabledSkills` · `skillDirectories` · `ignoredSkillsLocations`	Wyłączone skills · dodatkowe katalogi · katalogi pomijane
`enabledPlugins` · `extraKnownMarketplaces`	Deklaratywna instalacja wtyczek · dodatkowe marketplace'y
`proxyUrl` · `proxyKerberosServicePrincipal`	Proxy HTTP(S) (nadpisywane przez HTTP_PROXY/HTTPS_PROXY) · SPN Kerberos
`remote` · `remoteExport` · `remoteSessions`	Synchronizacja sesji i zdalne sterowanie
`respectGitignore` · `shellShortcut` · `storeTokenPlaintext`	Pliki z .gitignore poza pickerem @ · skrót $ · token tekstem jawnym
`powershellFlags`	Windows: flagi PowerShell (domyślnie -NoProfile, -NoLogo)

Ustawienia sandboksa [EXP]

Klucz	Znaczenie (domyślnie)
`sandbox.enabled` · `sandbox.allowBypass`	Sandbox włączony (false) · możliwość prośby o obejście (true)
`sandbox.auth.git` · `sandbox.auth.gh`	Wstrzykiwanie poświadczeń git / gh do sandboksa (true)
`sandbox.allowDevToolAccess`	Dostęp do ścieżek narzędzi deweloperskich
`sandbox.userPolicy.network.allowLocalNetwork`	Dostęp do sieci lokalnej, np. serwera deweloperskiego (true)
`sandbox.userPolicy.network.allowedHosts` · `blockedHosts`	Hosty dozwolone / blokowane (blokada wygrywa; *.example.com)
`sandbox.userPolicy.network.proxy`	Proxy dla ruchu z sandboksa (url, username, hasło w magazynie systemowym)
`sandbox.userPolicy.deniedPaths`	Ścieżki zabronione (nieobsługiwane w Windows)
`sandbox.userPolicy.seatbelt.keychainAccess`	macOS: dostęp do pęku kluczy (false)

Kluczowe ustawienia: interfejs (1/2)

Klucz	Znaczenie (domyślnie)
`theme`	`default, github (domyślny), dim, high-contrast, colorblind`
`banner · editorMode`	Baner: always/once/never · normal/vim
`footer.*`	`Elementy stopki: showModelEffort, showDirectory, showBranch, showContextWindow, showQuota, showAgent, showAiUsed, showCodeChanges, showUsername, showSandbox, showYolo, showCustom`
`statusLine`	Własny pasek stanu: type: "command", command, padding, refreshInterval
`tabs.enabled · tabs.hide · tabs.sort`	Pasek zakładek: włączenie, ukrywanie, kolejność
`sidebar · sidebar.*`	Pasek boczny sesji i jego opcje
`transcriptView`	default lub concise (grupowanie aktywności narzędzi)
`streamerMode`	Ukrywa nazwy modeli preview, limity i znaczniki czasu (pokazy, nagrania)

```json
{
  "statusLine": { "type": "command", "command": "~/.copilot/statusline.sh", "refreshInterval": 10 },
  "footer": { "showQuota": true, "showAiUsed": true, "showBranch": true }
}
```

Kluczowe ustawienia: interfejs (2/2) i inne

Klucz	Znaczenie (domyślnie)
`mouse` · `copyOnSelect` · `scrollbar`	Mysz · kopiowanie zaznaczenia · pasek przewijania
`beep` · `beepOnSchedule`	Sygnał dźwiękowy, gdy potrzebna uwaga · po zadaniu z harmonogramu
`terminalNotifications` · `terminalProgress` · `updateTerminalTitle`	Powiadomienia OSC 777 · postęp OSC 9;4 · tytuł okna = bieżący zamiar
`taskbarPresence`	Windows: sesje na pasku zadań
`compactPaste` · `renderMarkdown` · `renderHexColors` · `stream`	Zwijanie długich wklejeń · Markdown · próbki kolorów · strumieniowanie
`screenReader` · `showTimestamps` · `showTipsOnStartup` · `pinnedPrompts`	Czytnik ekranu · znaczniki czasu · porady · przypięty prompt
`keepAlive` · `logLevel`	Blokada uśpienia (off) · poziom logowania (default)
`autoUpdate` · `autoUpdatesChannel`	Auto-aktualizacje (true) · kanał stable/prerelease
`commandHistoryMaxSize` · `companyAnnouncements`	Historia poleceń (50) · komunikaty firmowe przy starcie
`worktreeBaseRef` · `worktreePathTemplate` · `experimental`	Worktree · tryb eksperymentalny (false)

Ustawienia zarządzane (MDM) — lokalizacje i klucze

Platforma	Lokalizacja
macOS	Profil MDM com.github.copilot lub /Library/Application Support/GitHub Copilot/managed-settings.json
Windows	Rejestr HKLM\SOFTWARE\Policies\GitHub Copilot lub %ProgramFiles%\GitHub Copilot\managed-settings.json
Linux	`/etc/github-copilot/managed-settings.json`

Obsługiwane klucze	`model`, `autoTier`, `permissions`, `sandbox`, `allowedMcpServers`, `deniedMcpServers`, `enabledPlugins`, `extraKnownMarketplaces`, `strictKnownMarketplaces`, `forceLoginOrgs`, `forceRemoteSettingsRefresh`, `remoteControl`, `shellShortcut`, `telemetry`, `policyHelper` (jeszcze nieaktywny)

Reguły zarządzane — przykład

Zasady
`Rodziny reguł: Bash(…), Shell(…), PowerShell(…), Read(…), Edit(…), Write(…), Domain(…)`
Pierwszeństwo: deny > ask > allow; gdy jakakolwiek lista jest ustawiona, operacja niepasująca do żadnej → ask
Edit/Write obejmują też przekierowania w shellu (>, >>) i sed -i
Wbudowany serwer GitHub MCP jest zawsze dozwolony; deniedMcpServers zawsze wygrywa

```json
{
  "model": "auto",
  "permissions": {
    "disableBypassPermissionsMode": "disable",
    "deny":  ["Shell(rm -rf *)", "Domain(*.evil.example)"],
    "ask":   ["Shell(git push *)"],
    "allow": ["Read(**)"]
  },
  "allowedMcpServers": [
    { "serverUrl": "https://mcp.example.com/*" },
    { "serverCommand": ["npx", "-y", "@example/mcp-server"] }
  ],
  "deniedMcpServers": [ { "serverName": "untrusted-server" } ],
  "forceLoginOrgs": ["my-org"]
}
```

Logi, diagnostyka i telemetria

Opcja / polecenie	Działanie
`--log-dir=KATALOG`	Katalog logów (domyślnie ~/.copilot/logs/)
`--log-level=POZIOM`	`none, error, warning, info, debug, all, default`
`/diagnose [PROMPT]`	Analiza logu sesji pod kątem błędów
`/collect-debug-logs, --collect-debug-logs`	Zebranie logów diagnostycznych (dla wszystkich od 1.0.84)
`copilot help logging · copilot help monitoring`	Pomoc o logowaniu i monitoringu (OpenTelemetry)
Polityka telemetry	`Eksport OpenTelemetry: enabled, endpoint, protocol, headers, resourceAttributes, captureContent, serviceName`
`/feedback (/bug)`	Zgłoszenie problemu

OpenTelemetry — obserwowalność pracy agentów

Zmienna	Znaczenie
`COPILOT_OTEL_ENABLED`	Włącza OTel (domyślnie wyłączone)
`OTEL_EXPORTER_OTLP_ENDPOINT`	Adres kolektora OTLP — ustawienie włącza OTel
`COPILOT_OTEL_EXPORTER_TYPE` · `COPILOT_OTEL_FILE_EXPORTER_PATH`	otlp-http (domyślnie) lub file · zapis do pliku JSONL
`OTEL_EXPORTER_OTLP_PROTOCOL` · `OTEL_EXPORTER_OTLP_HEADERS`	http/json lub http/protobuf · nagłówki uwierzytelnienia
`OTEL_SERVICE_NAME` · `OTEL_RESOURCE_ATTRIBUTES`	Nazwa usługi (domyślnie github-copilot) · dodatkowe atrybuty
`OTEL_INSTRUMENTATION_GENAI_CAPTURE_MESSAGE_CONTENT`	Pełna treść promptów i odpowiedzi (domyślnie false)

```bash
OTEL_EXPORTER_OTLP_ENDPOINT=https://collector.example.com:4318 \
OTEL_EXPORTER_OTLP_HEADERS="Authorization=..." copilot
COPILOT_OTEL_FILE_EXPORTER_PATH=/tmp/copilot-otel.jsonl copilot   # plik (offline / CI)
```

Zmienne i opcje zaawansowane

Zmienna / opcja	Działanie
`COPILOT_EDITOR`	Edytor (po $VISUAL i $EDITOR; domyślnie vi)
`COPILOT_ENABLE_HTTP2`	Transport HTTP/2 (domyślnie HTTP/1.1)
`USE_BUILTIN_RIPGREP` · `USE_TGREP` · `TGREP_FILE_COUNT_THRESHOLD`	Wbudowany ripgrep · indeksowane wyszukiwanie tgrep · próg plików (10 000 Windows / 50 000 inne)
`COPILOT_MCP_TOOL_CACHE`	false = bez migawek narzędzi MCP
`COPILOT_PROMPT_FRAME`	Ozdobna ramka wokół promptu (1/0)
`--extension-sdk-path`	Lokalny @github/copilot-sdk dla rozszerzeń **[EXP]**
`--mcp-github-auth`	Ograniczenie uwierzytelnienia GitHub do zatwierdzonych serwerów MCP (1.0.90)
`--server, --ahp-host`	Tryby hostowania sesji (serwer SDK, host AHP) — także objęte polityką firmy

Funkcje eksperymentalne i w preview (poza zakresem RTM)

Funkcja	Status	Włączenie
Sandbox lokalny (/sandbox, --sandbox, --no-sandbox)	Eksperymentalna w CLI, publiczny preview; Windows: Insiders	`--experimental` / `/experimental on`
`/every, /after`	Eksperymentalne	tryb eksperymentalny
Rozszerzenia CLI (/extensions) i dynamiczne workflowy	Eksperymentalne w CLI (workflowy: publiczny preview)	tryb eksperymentalny
/mcp search (rejestr MCP)	Eksperymentalne wg dokumentacji how-to	tryb eksperymentalny
Sandbox w chmurze (copilot --cloud)	Eksperymentalny	--cloud --experimental + polityka **[ORG]**
/diff — automatyczny diff gałęzi	Eksperymentalne	tryb eksperymentalny
Serwer ACP	Publiczny preview	`--acp`
Limity AI Credits sesji · Copilot Memory	Publiczny preview	/limits · polityka + ustawienia

Część 13

13

Tryb programistyczny, CI/CD i serwer ACP

Copilot CLI jako komponent skryptów, pipeline'ów i edytorów

Tryb nieinteraktywny: -p

`-p, --prompt=PROMPT`	wykonuje prompt i kończy działanie (bez pytań o zgodę)
`-i, --interactive=PROMPT`	otwiera sesję interaktywną i od razu wysyła prompt
`-s, --silent`	tylko odpowiedź agenta, bez statystyk — idealne do potoków
`--output-format=text|json`	json = JSONL (jeden obiekt JSON na linię) do parsowania
`--stream=on|off`	strumieniowanie odpowiedzi
Prompt może też przyjść potokiem na standardowe wejście	
Podsumowanie po zakończeniu zawiera copilot --resume=SESSION-ID — można kontynuować interaktywnie	

```bash
copilot -p "Co robi ten projekt?" -s --model claude-haiku-4.5
copilot -p "Napraw wyścig w puli workerów" --model gpt-5.3-codex --allow-tool='write, shell'
copilot -p "Przejrzyj ostatni commit" --allow-tool='shell' --agent code-review
copilot -p "Wypisz moduły bez testów" --output-format json > events.jsonl
```

Najważniejsze opcje do automatyzacji

Opcja	Po co w automatyzacji
`--allow-tool / --deny-tool`	Precyzyjne uprawnienia (brak interaktywnych zgód)
`--available-tools / --excluded-tools`	Zawężenie zestawu narzędzi widocznych dla modelu
`--allow-all-tools / --allow-all`	Pełne zaufanie — tylko w izolowanych, jednorazowych runnerach
`--no-ask-user`	Agent nigdy nie czeka na odpowiedź człowieka
`--model · --effort`	Przypięcie modelu i wysiłku rozumowania (powtarzalność)
`--agent=NAZWA`	Wyspecjalizowany agent niestandardowy
`--add-dir · --attachment`	Dodatkowe katalogi · pliki/obrazy do pierwszego promptu
`--secret-env-vars`	Maskowanie sekretów w logach
`--share · --share-gist`	Transkrypcja jako artefakt audytowy
`--enable-memory`	Pamięć w trybie -p (domyślnie wyłączona)
`--fleet · --autopilot · --max-autopilot-continues · --max-ai-credits`	Równoległość, autonomia i budżet

Zmienne środowiskowe dla automatyzacji

Zmienna	Działanie
`COPILOT_GITHUB_TOKEN (→ GH_TOKEN → GITHUB_TOKEN)`	Token (fine-grained PAT z „Copilot Requests”)
`COPILOT_ALLOW_ALL`	Wszystkie uprawnienia; dokładnie true = też zaufanie do katalogu i jego hooków/MCP
`COPILOT_MODEL`	Model dla procesu
`COPILOT_HOME`	Osobny katalog konfiguracji (np. czysta konfiguracja per job)
`COPILOT_AUTO_UPDATE=false`	Przypięta wersja CLI
`COPILOT_TASK_WAIT_TIMEOUT_SECONDS`	Ile czekać w -p na agentów i polecenia w tle przed wyjściem (domyślnie 600; 0 = nie czekaj)
`GITHUB_COPILOT_PROMPT_MODE_REPO_HOOKS=true`	Ładuje hooki repozytorium w -p
`GITHUB_COPILOT_PROMPT_MODE_WORKSPACE_MCP=true`	Ładuje serwery MCP z repozytorium w -p
`GITHUB_COPILOT_PROMPT_MODE_EXTENSIONS=true`	Ładuje rozszerzenia projektu w -p i workflow run
`COPILOT_PLAN_THEN_AUTOPILOT=1`	„Plan, potem autopilot” bez opcji wiersza poleceń

Przykład: Copilot CLI w GitHub Actions (poglądowy)

```yaml
name: copilot-review
on: pull_request
jobs:
  review:
    runs-on: ubuntu-latest
    permissions:
      contents: read
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0
      - uses: actions/setup-node@v4
        with:
          node-version: 22
      - run: npm install -g @github/copilot
      - name: Przegląd zmian przez Copilot
        env:
          COPILOT_GITHUB_TOKEN: ${{ secrets.COPILOT_PAT }}
          COPILOT_AUTO_UPDATE: "false"
        run: |
          copilot -p "Przejrzyj zmiany tej gałęzi względem origin/main. Wypisz ryzyka i brakujące testy." \
            -s --no-ask-user --model claude-sonnet-4.6 \
            --available-tools='view, glob, grep, bash' \
            --allow-tool='shell(git:*)' --deny-tool='shell(git push)' \
            --share=copilot-review.md
      - uses: actions/upload-artifact@v4
        with:
          name: copilot-review
          path: copilot-review.md
```

copilot workflow run — dynamiczne workflowy

Element	Opis
`copilot workflow run NAZWA`	Uruchamia workflow zarejestrowany przez rozszerzenie (nazwa, nie ścieżka) i czeka na zakończenie
`--args JSON / --args @plik.json`	Dane wejściowe workflow (domyślnie {})
`--result-file ŚCIEŻKA`	Wynik jako JSON do pliku (zapisywany tylko po sukcesie)
`-s, --silent`	Bez komunikatów postępu
`--output-format text|json`	json = JSONL; ostatni rekord type: workflow.result ze statusem completed|halted|paused|cancelled|error
Kody wyjścia	0 sukces · 1 nieukończony lub błąd · 2 brak workflow / złe argumenty · 130 przerwanie (SIGINT/SIGTERM)
`/workflows`	Podgląd przebiegów, faz, agentów i postępu (plan z rozliczaniem tokenowym)

Przepisy skryptowe

```bash
# 1. Opis zmian do commita (tylko odczyt + git)
copilot -p "Zaproponuj wiadomość commita w konwencji Conventional Commits dla zmian w staging" \
  -s --available-tools='bash, view' --allow-tool='shell(git:*)' --deny-tool='shell(git push)'

# 2. Raport JSONL do dalszej analizy
copilot -p "Znajdź martwy kod w src/" --output-format json --allow-tool='shell(rg:*)' > raport.jsonl

# 3. Równoległe testy jednostkowe z budżetem i transkrypcją
copilot --fleet -p "Dopisz brakujące testy jednostkowe w src/services" \
  --allow-tool='write, shell(npm test)' --max-ai-credits=20 --share=fleet-tests.md

# 4. Kontynuacja pracy interaktywnie po zadaniu wsadowym
copilot --resume=<SESSION-ID>
```

Serwer ACP (Agent Client Protocol) — publiczny preview

Element	Opis
`copilot --acp / --acp --stdio`	Serwer ACP na stdio (domyślnie) — klient uruchamia CLI jako proces potomny; zalecane dla IDE
`copilot --acp --port 3000`	Serwer TCP na 127.0.0.1; wiele połączeń, niezależny cykl życia
Opcje serwera	--available-tools, --excluded-tools, --effort — obowiązują dla każdej sesji
Polecenia slash	Wysyłane jako zwykły tekst promptu; informacyjne (/usage, /context) bez modelu, akcyjne (/plan, /review) uruchamiają agenta
Lista poleceń	Powiadomienie available_commands_update (pełna lista, także skills jako /NAZWA)
Niedostępne przez ACP	Polecenia z UI pełnoekranowym: /diff, /resume, /theme, /settings, /login, /help, /tasks, /undo
Ograniczenia	Brak --fleet; polityki zarządzane obowiązują także w ACP; BYOK działa bez logowania do GitHub

```bash
copilot --acp --port 3000 --effort=max --available-tools="bash,view"
```

```typescript
// klient: npm install @agentclientprotocol/sdk
const proc = spawn("copilot", ["--acp", "--stdio"], { stdio: ["pipe", "pipe", "inherit"] });
```

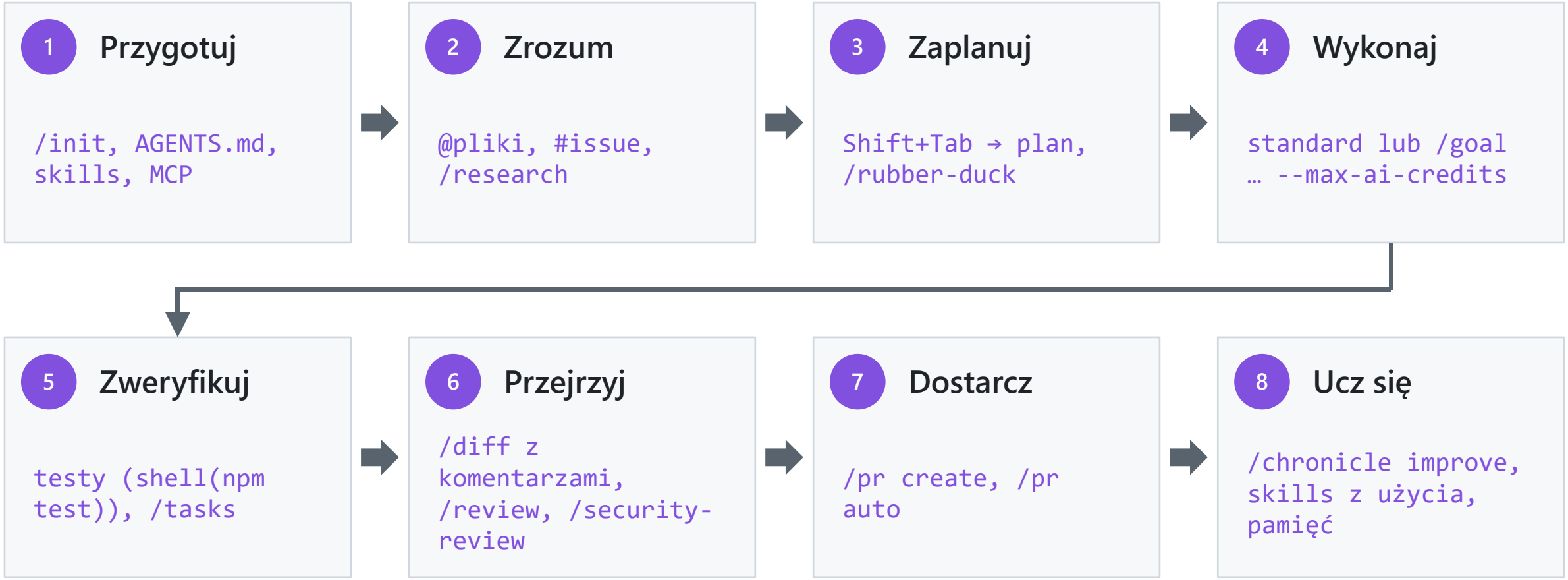
Część 14

14

Wzorce pracy agentycznej

Jak łączyć opcje w skuteczny proces

Przepływ referencyjny: od zadania do pull requesta



Wzorzec 1: plan z człowiekiem, wykonanie przez autopilota

```text
$ copilot -n auth-jwt --model claude-opus-5.5
> /plan Zastąp sesje cookie tokenami JWT w module auth; zachowaj zgodność API
> /rubber-duck Oceń plan: ryzyka migracji i brakujące testy
  ... poprawki planu w rozmowie ...
> /model claude-sonnet-4.6
> /goal Wdróż zatwierdzony plan; wszystkie testy muszą przechodzić --max-ai-credits 10
> /diff        # komentarze do linii → Enter
> /review
> /pr create
```

Wzorzec 2: równoległość bez konfliktów

Osobne zadania = osobne worktree i sesje — agenci nie nadpisują sobie plików
Pasek boczny sesji: statusy ! i ? pokazują, kto czeka na człowieka
/fleet dla niezależnych części jednego zadania w jednym drzewie
/delegate dla zadań długich lub tła — praca w chmurze kończy się pull requestem
/fork — eksperyment z alternatywnym podejściem bez utraty bieżącego wątku

```bash
copilot -w feature-search -n search -i "Zaimplementuj wyszukiwanie pełnotekstowe wg #311"
copilot -w fix-1234 -n bugfix  -i "#1234 napraw błąd i dodaj test regresji"
```

Wzorzec 3: obrona warstwowa (guardrails)

Warstwa	Mechanizm	Przykład
Instrukcje	AGENTS.md, .github/instructions	„Nigdy nie commituj sekretów; migracje tylko przez skrypt X"
Widoczność narzędzi	`--available-tools, --excluded-tools`	Agent analityczny bez narzędzi zapisu
Uprawnienia	`--allow-tool, --deny-tool, /permissions`	shell(git:*) bez shell(git push)
Hooki	`preToolUse`	Blokada poleceń spoza listy, audyt każdego wywołania
Izolacja	sandbox **[EXP]**, worktree, kontener	Brak dostępu do sieci i katalogu domowego
Sekrety	`--secret-env-vars, osobny COPILOT_HOME`	Maskowanie kluczy w logach
Polityka firmy	managed settings / MDM	`disableBypassPermissionsMode: "disable"`
Przegląd	`/diff, /review, /security-review`	Człowiek zatwierdza wynik

Wzorzec 4: zespół uczy agenta

/init + przegląd i commit instrukcji — wspólna baza wiedzy repozytorium
Pamięć repozytorium — agent zapamiętuje konwencje odkryte w trakcie pracy
/chronicle tips i /chronicle improve — wskazówki na podstawie historii sesji
/chronicle skills create|review|status — propozycje skills repozytorium z obserwowanego użycia
Skills i agenci w .github/skills, .github/agents — powtarzalne procedury jako kod
Wtyczki i marketplace — dystrybucja standardów do wielu repozytoriów

Pułapki i jak ich uniknąć

Pułapka	Jak uniknąć
--yolo w nieznanym repozytorium	Najpierw tryb domyślny i przegląd hooków/MCP/instrukcji repo
COPILOT_ALLOW_ALL=true w CI z cudzym kodem	Wąskie --allow-tool, osobny COPILOT_HOME, izolowany runner
Klasyczny PAT (ghp_...) w CI	Fine-grained PAT z uprawnieniem „Copilot Requests”
Jedna sesja do wszystkiego	Jedno zadanie = jedna sesja; /new, /compact
Autopilot bez budżetu	`--max-ai-credits, --max-autopilot-continues, /limits`
Brak --no-ask-user w CI	Agent czeka na odpowiedź, której nikt nie udzieli
Równoległe sesje w jednym katalogu	`--worktree, /fork worktree`
„Agent nie widzi mojego MCP/skilla”	/env, zaufanie folderu, zmienne GITHUB_COPILOT_PROMPT_MODE_* w -p
Niedoszacowany koszt	/usage, tańszy model do implementacji, Auto

Ściągawka: polecenia slash

`/plan`	plan przed kodem
`/goal (/autopilot)`	cel dla autopilota
`/fleet`	subagenci równolegle
`/tasks`	zadania w tle
`/delegate`	praca w chmurze → PR
`/pr`	pull request: create, fix, auto
`/diff`	przegląd zmian z komentarzami
`/review · /security-review`	przeglądy
`/rubber-duck`	druga opinia
`/research`	głębokie badanie
`/undo (/rewind)`	cofanie tury
`/ask (/btw)`	pytanie poboczne

`/model`	model, rozumowanie, kontekst
`/permissions`	tryb uprawnień
`/context · /compact`	okno kontekstu
`/init · /instructions`	instrukcje
`/mcp · /skills · /plugin · /agent`	rozszerzenia
`/resume · /fork · /worktree`	sesje
`/share`	eksport sesji
`/remote`	sterowanie z telefonu
`/usage · /limits`	koszty
`/settings`	ustawienia
`/env · /diagnose`	diagnostyka
`/help`	pełna lista

Ściągawka: opcje wiersza poleceń

praca interaktywna
-i "prompt" — sesja z promptem startowym
`--plan, --autopilot, --mode=…`
`--model=…, --effort=…, --context=…`
`-r/--resume, --continue, -n NAZWA`
`-w/--worktree[=NAZWA], -C KATALOG`
`--agent=NAZWA`
`--add-dir=ŚCIEŻKA`
`--remote, --connect`
`--experimental`

automatyzacja i kontrola
`-p "prompt", -s, --output-format=json`
`--allow-tool, --deny-tool`
`--available-tools, --excluded-tools`
`--allow-url, --deny-url`
`--no-ask-user`
`--max-ai-credits, --max-autopilot-continues`
`--secret-env-vars`
`--share, --share-gist`
`--additional-mcp-config`
`--fleet`

Ćwiczenia laboratoryjne

Lab	Zadanie	Kluczowe opcje
1. Start	Instalacja, logowanie, /init, przegląd instrukcji	`copilot login`, `/init`, `/env`
2. Kontekst	Zadanie z @plikiem i #issue; porównanie z --no-custom-instructions	`@`, `#`, `/context`, `/compact`
3. Plan → autopilot	Funkcja od planu do PR z budżetem	`/plan`, `/rubber-duck`, `/goal --max-ai-credits`, `/pr create`
4. Bezpieczeństwo	Skrypt -p z minimalnymi uprawnieniami + hook blokujący	`--allow-tool`, `--deny-tool`, `preToolUse`
5. Rozszerzenia	Własny skill i agent .agent.md + serwer MCP	`/skills`, `/agent`, `copilot mcp add`
6. Równoległość	Dwie sesje w worktree + /fleet	-w, pasek boczny, /tasks
7. CI	Workflow GitHub Actions z przeglądem PR	`-p`, `-s`, `--share`, `COPILOT_GITHUB_TOKEN`

Podsumowanie

Copilot CLI to pełne środowisko agentyczne: tryby pracy, narzędzia, subagenci, praca w chmurze
Jakość wyniku = kontekst (instrukcje, MCP, skills) × kontrola (plan, uprawnienia, review)
Autonomię zwiększaj stopniowo: standard → plan → autopilot z budżetem i guardrailami
Te same mechanizmy działają interaktywnie i w automatyzacji (-p, CI, ACP)
Rozszerzalność (skills, agenci, hooki, wtyczki) zamienia wiedzę zespołu w powtarzalne procesy
CLI zmienia się co kilka dni — /changelog i copilot help to źródła prawdy

Źródła

GitHub Docs — Copilot CLI reference: command reference, programmatic reference, configuration directory, plugin reference, ACP server (docs.github.com/en/copilot/reference/copilot-cli-reference)
GitHub Docs — hooks reference i custom agents configuration (docs.github.com/en/copilot/reference)
GitHub Docs — strony how-to i koncepcyjne Copilot CLI: autopilot, /fleet, /research, rubber duck, zarządzanie kontekstem, sesje i worktree, /pr, zdalne sterowanie, skills, MCP, LSP, rozszerzenia, sandbox, rozliczenia
Repozytorium github/copilot-cli — README, changelog.md, releases (wersja 1.0.92 z 5.10.2026)
GitHub Changelog: „GitHub Copilot CLI is now generally available" (25.02.2026), „Updates to GitHub Copilot billing and plans" (01.06.2026), „Copilot CLI: New terminal interface is generally available" (23.06.2026)
Stan wiedzy: 6 października 2026